



Administratorhandbuch

Browser Policy Manager 0.9.3

Inhalt

Systemanforderungen und Lieferumfang

- Mindestsystemanforderungen für BPM
- Die Bereitschaft einer Einzelknoten-Quellbereitstellung bewerten
- HA- und Produktionsgrenzen dokumentieren

BPM aus dem Quellcode unter Linux bereitstellen

- Einen Linux-Host für die Quellbereitstellung vorbereiten
- Das Auschecken der Linux-Quelle einrichten
- Die Laufzeiteinstellungen der Linux-Quelle konfigurieren
- Eine Linux-Quellbereitstellung prüfen
- BPM aus dem Quellcode unter Ubuntu 26.04 LTS installieren
- BPM aus dem Quellcode unter Debian 13.5 installieren
- BPM aus dem Quellcode unter Fedora Linux 44 installieren
- BPM von der Quelle unter Linux Mint 22.3 installieren
- BPM aus dem Quellcode im stabilen Manjaro-Zweig installieren

BPM aus dem Quellcode mit Windows WSL bereitstellen

- Windows 10/11 für die WSL-Quellbereitstellung vorbereiten
- Das Auschecken der BPM-Quelle innerhalb der WSL einrichten
- WSL-Laufzeit und Browserzugriff konfigurieren
- Die Bereitstellung der BPM-Quelle über WSL prüfen

Eine BPM-Quellbereitstellung betreiben und aktualisieren

- DevOps-Konfigurationsquellen prüfen
- Speicherung, Protokolle und Sicherungsnachweise planen
- Netzwerk, CORS und Sicherheitsbeschränkungen prüfen
- Betriebsgrenzen einer Quellbereitstellung dokumentieren
- Nachweise vor einer Quellaktualisierung vorbereiten
- Quellrevision und Abhängigkeiten aktualisieren
- Migrationen und Prüfungen der Quellaktualisierung durchführen
- Das Update prüfen und Rollback-Stopp-Bedingungen anwenden
- BPM-Zustand und -Bereitschaft prüfen

BPM über die API integrieren

- Integrationszielgruppe und API-Grenze
- Unterstützte Integrationsmuster
- API-Konventionen

- Aktuelle API-Einschränkungen und Sicherheitsbeschränkungen
- DevOps-API-Beispiele und Umgebungsvorlagen wiederverwenden
- Profillebenszyklusdaten synchronisieren
- Profile archivieren, wiederherstellen, dauerhaft löschen oder zurücksetzen
- Firefox policies.json über die API importieren
- Die kanonische Firefox policies.json über die API exportieren
- Die Firefox-Kandidatendatei policies.json über die API validieren

Lebenszyklusabläufe ausführen und Fehler beheben

- Ein Pull-, Vergleichs- und Aktualisierungsszenario für ein Steuerungsprodukt ausführen
- Ein Import-, Prüfungs-, Export- und Compliance-Übergabeszenario ausführen
- BPM-Profil aus einem externen Steuerungsprodukt abrufen und prüfen
- Eine Profilaktualisierung mit expected_revision validieren und anwenden
- Import, Prüfung, Export und Übergabe von Compliance-Metadaten durchführen
- Start eines Steuerungsprodukts mit BPM-Health- und Readiness-Prüfungen
- Wiederherstellung nach Integrationsfehlern und Dokumentation von Prüfnachweisen

Fehler beheben und Betriebsbereitschaft bewerten

- Fehlerbehebung bei fehlgeschlagenem Start und nicht erreichbaren Sonden
- Schema-Cache- und API-Validierungsfehler beheben
- Fehlerbehebung bei Firefox-Import- und Exportfehlern
- Datenbank- und Speicherprobleme beheben
- Fehlerbehebung bei WSL-Netzwerken und veralteten Abhängigkeiten
- Nicht verfügbares Dokumentationsportal beheben
- Netzwerkexposition und Proxy-Bereitschaft prüfen
- Überwachungseingänge, Sicherungsnachweise und Aktualisierungsfenster planen

BPM-Dokumentationsassistent

- Den Dokumentationsassistenten in BPM 0.9.3 betreiben
- Die Grenze des Dokumentationsassistenten in BPM 0.9.3 pflegen

Systemanforderungen und Lieferumfang

Mindestsystemanforderungen für BPM

Verwenden Sie diese Mindestwerte für den unterstützten BPM-Quellcode-Workflow in Version 0.9.3.

BPM-Grundsystem

Führen Sie BPM über den unterstützten Linux-x86_64-Quellcode-Workflow aus: Ubuntu 26.04 LTS, Debian 13.5, Fedora Linux 44, Linux Mint 22.3 oder den aktuellen stabilen Manjaro-Zweig. Windows 10 und Windows 11 werden nur als WSL-2-Host für diesen Linux-Workflow unterstützt. BPM stellt keinen nativen Windows-Dienst oder Installer bereit. Erforderlich sind CPython 3.14, zwei CPU-Kerne, 4 GiB RAM und 4 GB beschreibbarer freier Speicher für Checkout, virtuelle Umgebung, Datenbank, Protokolle und Dokumentationsbuild.

Browser, Netzwerk und Zugriff

Verwenden Sie einen aktuellen Desktopbrowser mit aktiviertem JavaScript und HTTPS für die BPM-Weboberfläche und das Dokumentationsportal. Die unterstützten Firefox-Schemata beschreiben die Kompatibilität von Richtliniendaten und verlangen nicht, BPM in Firefox auszuführen. HTTPS und sudo sind nur für die Installation von Betriebssystempaketen oder Quellabhängigkeiten erforderlich. Der Prozess benötigt einen normalen Linux-Benutzer mit Schreibzugriff auf Checkout und konfigurierte Datenverzeichnisse; führen Sie die Anwendung nicht allein für den Betrieb als root aus.

Dokumentationsassistent in 0.9.3

Der Dokumentationsassistent ist als Leseroberfläche verfügbar, doch Version 0.9.3 liefert kein lokales Modell, keinen RAG-Index, keine Embeddings, keine generierten Antworten, keine Zitate und keinen Zugriff auf externe Quellen. Daher entstehen keine zusätzlichen Anforderungen an CPU, RAM, Speicher, GPU, NPU, Cloudkonto oder Netzwerk über den BPM-Grundworkflow hinaus.

Jede gesendete Frage zeigt den lokalisierten Hinweis zur Schulung des lokalen Modells. Nutzen Sie für aktuelle Produktinformationen die unabhängige deterministische Dokumentationsuche.

Die Bereitschaft einer Einzelknoten-Quellbereitstellung bewerten

Entscheiden Sie, ob die aktuelle, von der Quelle ausgeführte BPM-Instanz für den kontrollierten Einzelknotenbetrieb bereit ist, und benennen Sie die Beweise, die diese Entscheidung stützen.

Die Bereitstellung der Linux- oder Windows 10/11-Quelle über die WSL ist abgeschlossen, Abhängigkeiten sind installiert und die beabsichtigten Werte für `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR` und `BPM_SCHEMA_CACHE_DIR` sind bekannt.

BPM kann für den Einzelknoten-Quellenbetrieb im aktuellen Zustand vorbereitet werden. Bei dieser Bewertung handelt es sich nicht um ein Zertifikat zur Produktionshärtung, kein HA-Design und auch nicht um eine paketierte Serviceabnahme.

1. Zeichnen Sie die genaue Quell- und Laufzeitkonfiguration auf.

Erfassen Sie `git rev-parse --short HEAD`, die BPM-Version, `BPM_HOST`, `BPM_PORT`, `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR`, `BPM_SCHEMA_CACHE_DIR` und ob der Prozess mit `make dev` oder einem direkt besessenen `uvicorn app.main:app`-Befehl gestartet wird.

2. Führen Sie Funktions- und Bereitschaftsprüfungen von derselben Netzwerkadresse aus, die Benutzer verwenden.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
make dev
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Zeichnen Sie das Ergebnis als `GET /health`- und `GET /health/ready`-Beweis auf. Bestehende Prüfungen beweisen nur die Lebendigkeit und Bereitschaft des Quellprozesses.

3. Überprüfen Sie die Daten und Dokumentationsnachweise, bevor Sie die Instanz freigeben.

Bestätigen Sie, dass `data/bpm.db` die vorgesehene Datenbank ist, dass mindestens ein Profil als `policies.json` exportiert werden kann und dass `/help/` das erstellte Dokumentationsartefakt oder einen dokumentierten Nichtverfügbarkeitsstatus öffnet.

4. Klassifizieren Sie die Instanz als Quelloperation im aktuellen Status.

WARNUNG:

Kennzeichnen Sie die Instanz nicht als gepackt, gehärtet, Lastausgleich oder hochverfügbar. Wenn externe Kontrollen diese Eigenschaften hinzufügen, erfassen Sie sie als externe Verantwortlichkeiten und beschränken Sie die BPM-Beweise auf den Quellbetrieb mit einem Knoten.

In der Bereitschaftsnotiz heißt es, was heute sicher aus der Quelle erledigt werden kann, welche Prüfungen bestanden wurden und welche Entscheidungen auf die Verteilung und Laufzeitunterstützung warten müssen.

Verwenden Sie das Thema „Netzwerkfreigabe und Proxy-Bereitschaft“, bevor Sie die Instanz über eine lokale Administrator-Workstation oder einen kontrollierten Laborhost hinaus verfügbar machen.

HA- und Produktionsgrenzen dokumentieren

Geben Sie an, welche Produktions-, HA-, Reverse-Proxy-, Wiederherstellungs- und Service-Management-Funktionen heute fehlen und was für spätere Entwurfsarbeiten vorbereitet werden kann.

Für die aktuelle Quellausführungsinstanz sind Hinweise zu Bereitschaft, Netzwerkgefährdung, Sicherung/Export, Überwachung und Aktualisierungsfenster vorhanden.

In diesem Thema wird die Grenze zwischen dem aktuellen Einzelknoten-Quellbetrieb und der zukünftigen Produktionsarchitektur aufgezeichnet. Es wird bewusst explizit darauf hingewiesen, was geplant, aber noch nicht umgesetzt ist.

1. Benennen Sie die unterstützte aktuelle Form.

Schreiben Sie, dass die BPM-Administrator-/DevOps-Dokumentation Quellbereitstellung, manuelle Konfiguration, Integritäts-/Bereitschaftsprüfungen, API-Beispiele, manuelle Sicherungsnachweise, Dokumentationsneuerstellungsprüfungen und Aktualisierungsfenster für einzelne Knoten unterstützt.

2. Name fehlender Dienst- und Proxyfunktionen.

Listen Sie verpackte Dienste, native Linux-Diensteinheiten, Windows-Dienste, offizielle Reverse-Proxy-Rezepte, TLS/Proxy-Rezepte, verwaltete Geheimnisse, Authentifizierungs-Gateway-Konfiguration und Produktionshärtung auf, die in der BPM-Quelldokumentation fehlen.

3. Name fehlt HA und Upgrade-Funktionen.

Listen Sie unterstütztes HA-Clustering, Multi-Knoten-Koordination, Anleitung zur Load-Balancer-Sitzungsaffinität, gemeinsame Speichersperre, Migration ohne Ausfallzeiten, fortlaufende Upgrades, automatisches Failover und offizielle Wiederherstellungsautomatisierung als zukünftige Arbeit auf, nicht als aktuelles Verhalten.

4. Wandeln Sie zukünftige Anforderungen in Design-Inputs um.

WARNUNG:

Platzieren Sie nicht mehrere Quellknoten hinter einem Load Balancer und nennen Sie das unterstützte Ergebnis HA. Erfassen Sie erforderliche RTO/RPO, Sicherungsüberprüfung, TLS-Besitz, Secrets-Besitz, Protokollierung, Überwachung und Upgrade-Fenstererwartungen als Eingaben für spätere Verteilungs-/Laufzeitaufgaben.

Der Grenzdatsatz verhindert nicht unterstützte Produktions-, HA- und Reverse-Proxy-Ansprüche und bewahrt gleichzeitig nützliche Vorbereitungsarbeiten für zukünftige Versionen.

Verwenden Sie diesen Datensatz während der manuellen Dokumentation QA und wenn zukünftige Installer-, Service-, Reverse-Proxy- und HA-Backlog-Elemente genehmigt werden.

BPM aus dem Quellcode unter Linux bereitstellen

Einen Linux-Host für die Quellbereitstellung vorbereiten

Bestätigen Sie die unterstützte Quellbereitstellungsgrenze, Hostvoraussetzungen und verzögerte Produktionsfunktionen, bevor Sie BPM aus einem Repository-Checkout installieren.

Verwenden Sie einen Linux-Host, auf dem Sie Python 3.14+, Git, Build-Tools und curl installieren können. Dieses Betriebsanleitung geht von einem Quell-Checkout und nicht von einem gepackten Installationsprogramm aus.

BPM kann zur Entwicklung, Überprüfung und kontrollierten administrativen Evaluierung aus dem Quellcode ausgeführt werden. Gepackte Installationsprogramme, native Serviceeinheiten, offizielle Systemd-Rezepte, Reverse-Proxy- und TLS-Rezepte, HA-Clustering, verwaltete Geheimnisse und Produktionshärtung werden in dieser Version nicht bereitgestellt und dürfen nicht als unterstütztes Bereitstellungsverhalten dargestellt werden.

1. Identifizieren Sie den Bereitstellungszweck und notieren Sie, dass es sich um eine quellbasierte Linux-Bereitstellung handelt.

Erwartetes Ergebnis: Der Betriebshinweis besagt, dass BPM von einem Git-Checkout gestartet wird und dass verteilungsspezifische Installationsprogramme zurückgestellt werden.

2. Hostvoraussetzungen installieren oder überprüfen.

Erforderliche Tools sind Git, Python 3.14+, pip, make, curl und eine Shell, die `source .venv/bin/activate` ausführen kann. Die Repository-Befehle nutzen die lokale virtuelle Umgebung und erfordern kein globales BPM-Paket.

3. Erstellen Sie ein Arbeitsverzeichnis, das von Produktionsdaten und Geheimnissen getrennt ist.

Halten Sie die Repository-Checkout-, `.venv`- und lokalen SQLite-Daten unter einem kontrollierten Pfad. Speichern Sie keine echten Geheimnisse in Dokumentationsbeispielen oder festgeschriebenen Dateien.

4. Bestätigen Sie, was absichtlich nicht abgedeckt ist, bevor Sie fortfahren.

WARNUNG:

Dieses Betriebsanleitung bietet kein gepacktes Installationsprogramm, keinen nativen Dienst, keine unterstützte Systemd-Einheit, kein Reverse-Proxy-Rezept, keine TLS-Beendigung, keine HA-Topologie, keinen verwalteten geheimen Speicher, keine Sicherungsautomatisierung oder keine Checkliste zur Produktionshärtung.

Die Host- und Betriebsannahmen sind für einen unterstützten BPM-Quell-Checkout-Arbeitsablauf bereit.

Fahren Sie mit dem Thema zum Auschecken der Linux-Quelle fort, bevor Sie Laufzeitvariablen konfigurieren.

Das Auschecken der Linux-Quelle einrichten

Klonen Sie das BPM-Repository, erstellen Sie die virtuelle Python-Umgebung, installieren Sie Entwicklungsabhängigkeiten und bereiten Sie das lokale Datenbankschema vor.

Der Linux-Host ist vorbereitet und der Administrator hat Zugriff auf die BPM-Repository-URL sowie den freigegebenen Zielzweig, das Ziel-Tag oder die Zielrevision.

Das Repository README definiert die unterstützte Quell-Setup-Sequenz. Verwenden Sie Repository-lokale Befehle, damit die Umgebung bei der späteren Fehlerbehebung mit festgeschriebenen Skripts verglichen werden kann.

1. Klonen Sie das Repository und gehen Sie zum Auschecken.

```
git clone <repository-url> browser-policy-manager
cd browser-policy-manager
git checkout <approved-0.9.1-ref>
```

Ersetzen Sie Platzhalter durch die genehmigte interne Repository-URL und die Release-Referenz.

2. Erstellen und aktivieren Sie die lokale Python-Umgebung.

```
python -m venv .env
source .env/bin/activate
python -m pip install --upgrade pip
```

Die virtuelle Umgebung bleibt innerhalb der Kasse und wird nicht festgeschrieben.

3. Installieren Sie BPM-Quellenabhängigkeiten aus dem Checkout.

```
pip install -e ".[dev]"
```

Durch die bearbeitbare Installation bleiben Laufzeitcode und Tests am aktuellen Quellbaum ausgerichtet.

4. Datenbankmigrationen für die ausgewählte Laufzeitdatenbank anwenden.

```
alembic upgrade head
```

WARNUNG:

Führen Sie Migrationen nur für die vorgesehene BPM-Datenbank aus. Für den Standard-Quellworkflow ist dies die lokale SQLite-Datenbank, keine Produktionsdienstdatenbank.

Der Checkout verfügt über eine aktivierte Python-Umgebung, installierte Abhängigkeiten und ein migriertes Datenbankschema.

Konfigurieren Sie die Laufzeitumgebung, bevor Sie den Anwendungsprozess starten.

Die Laufzeiteinstellungen der Linux-Quelle konfigurieren

Legen Sie die Datenbank-URL, den Host, den Port, den Neulademodus, CORS und den Dokumentations-Site-Pfad fest, der vom von der Quelle ausgeführten BPM-Prozess verwendet wird.

Die Checkout-Abhängigkeiten sind installiert und Migrationen wurden für die Datenbank angewendet, die Sie verwenden möchten.

BPM liest Umgebungsvariablen mit dem Präfix `BPM_` und unterstützt auch eine Repository-lokale `.env`-Datei. Die Standarddatenbank-URL ist `sqlite+aiosqlite:///./data/bpm.db`, die Daten in `data/bpm.db` speichert.

1. Erstellen Sie das lokale Datenverzeichnis und definieren Sie die Datenbank-URL.

```
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
```

Für wiederholbare Quellausführungen kann derselbe Wert in einer lokalen `.env`-Datei gespeichert werden, die nicht festgeschrieben wird.

2. Legen Sie Netzwerk- und Neuladewerte für den lokalen Quellprozess fest.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
```

Verwenden Sie Loopback für die lokale Überprüfung. Die Bindung an eine gemeinsam genutzte Schnittstelle ist eine betriebliche Entscheidung außerhalb dieser Quell-Betriebsanleitung-Grenze.

3. Überprüfen Sie die Einstellungen für CORS und die Dokumentationsseite.

Zu den Quellstandardwerten gehören `BPM_ENABLE_CORS`, `BPM_CORS_ALLOW_ORIGINS` und `BPM_DOCUMENTATION_SITE_DIR`. Behalten Sie die Erlaubnis von CORS nur für kontrollierte lokale Auswertungen bei und verweisen Sie `BPM_DOCUMENTATION_SITE_DIR` auf eine installierte statische Dokumentationsseite, sofern eine verfügbar ist.

4. Starten Sie BPM über das Repository-Skript.

```
make dev
```

Das Makefile führt `uvicorn app.main:app --reload --port 8000` aus. Lassen Sie das Vordergrundterminal geöffnet, damit Startmeldungen und Rückverfolgungen sichtbar sind.

BPM wird von der Quelle mit expliziten lokalen Laufzeiteinstellungen ausgeführt.

Überprüfen Sie den Gesundheitszustand, die Bereitschaft, das Produkt UI und die ersten administrativen Rauchkontrollen.

Eine Linux-Quellbereitstellung prüfen

Verwenden Sie Integritätstests, das Web UI, Protokolle und einen engen Testbefehl, um zu bestätigen, dass die von der Quelle ausgeführte BPM-Instanz verwendbar ist.

Der BPM-Prozess wird vom Repository-Checkout mit den beabsichtigten Einstellungen für BPM_DATABASE_URL, Host, Port und Dokumentationsseite ausgeführt.

Die aktuelle Anwendung stellt die Aktivität bei /health und die Bereitschaft bei /health/ready bereit. Diese Probes bestätigen nur die Reaktion des laufenden Prozesses; Sie beweisen keine Produktionshärtung, Firefox-Durchsetzung, Backup-Bereitschaft oder HA-Verhalten.

1. Überprüfen Sie die Lebendigkeit und Bereitschaft des Linux-Hosts.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Die erwarteten Antworten sind {"status": "ok"} und {"status": "ready", "ready": true}.

2. Öffnen Sie das Produkt UI und bestätigen Sie, dass die Bibliotheksroute antwortet.

Verwenden Sie `http://127.0.0.1:8000/profiles`. Die Route sollte die BPM-Schnittstelle laden oder einen beabsichtigten Anwendungsstatus anzeigen, keinen Server-Traceback.

3. Überprüfen Sie die Protokolle der Vordergrundanwendung.

Überprüfen Sie das Terminal, auf dem `make dev` oder `uvicorn app.main:app --reload --port 8000` ausgeführt wird. Erfassen Sie Startfehler, Migrationsfehler, Portkonflikte und Dokumentationsseitenwarnungen, ohne Geheimnisse in Tickets zu kopieren.

4. Dokumentieren Sie die Grenze des Prüfergebnisses.

WARNUNG:

Das Bestehen von Health-Probes, UI-Route und Vordergrundprotokollen bestätigt weder Produktionsreife noch Reverse-Proxy-Verhalten, verwaltete Sicherungen oder die Durchsetzung von Firefox-Richtlinien zur Laufzeit.

Der Administrator hat Beweise dafür, dass die Linux-Quellbereitstellung startet, Integritäts-/Bereitschaftsprüfungen beantwortet, die Route UI bedient und keine unmittelbaren Vordergrundprotokollblocker hat.

Wenn die Überprüfung fehlschlägt, bewahren Sie den Befehl, die Namen der Umgebungsvariablen, die Testantworten und die relevanten Protokollzeilen zur Fehlerbehebung zusammen. Erweitern Sie dieses Betriebsanleitung nicht um Anweisungen für verzögerte Installationsprogramme oder Produktionstopologien.

BPM aus dem Quellcode unter Ubuntu 26.04 LTS installieren

Vollständige Installation von BPM 0.9.1 mit den Python-3.14-Paketen der Distribution.

Erforderlich sind Ubuntu 26.04 LTS, sudo, HTTPS und eine freigegebene BPM-0.9.1-Git-Referenz.

Dieses Verfahren richtet einen kontrollierten Einzelknoten-Quellbetrieb ein, keinen Systemdienst.

1. Prüfen Sie Ubuntu und installieren Sie die Voraussetzungen.

```
. /etc/os-release
test "$ID" = "ubuntu" && test "$VERSION_ID" = "26.04"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make python3.14 python3.14-dev
python3.14-venv
export BPM_PYTHON=/usr/bin/python3.14
"$BPM_PYTHON" --version
```

2. Checken Sie die freigegebene Referenz aus.

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. Erstellen Sie die Umgebung, installieren Sie BPM und migrieren Sie die Datenbank.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. Starten Sie BPM im Vordergrund.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. Prüfen Sie Endpunkte, Oberfläche und Protokolle in einem zweiten Terminal.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Beachten Sie die Abbruchgrenze.

Drücken Sie bei einem Fehler `Ctrl+C`, bewahren Sie Ausgabe und Protokolle auf und führen Sie kein `alembic downgrade` aus.

Migration und Dokumentationsbau sind abgeschlossen; Endpunkte und Profilbibliothek antworten.

Die Befehlsfolge wurde in einem sauberen Ubuntu-26.04-Userspace-Container erfolgreich ausgeführt. Damit sind nativer Systemstart, Kernel, systemd, Desktop, Hardware, Host-Firewall und Produktivbetrieb nicht geprüft.

BPM aus dem Quellcode unter Debian 13.5 installieren

Vollständige Installation mit einer prüfsummenvalidierten CPython-3.14.6-Benutzerinstallation.

Erforderlich sind Debian 13.5, sudo, HTTPS, 4 GB freier Speicher und eine freigegebene BPM-Referenz.

Das System-Python 3.13 wird nicht ersetzt; Python 3.14.6 wird unter \$HOME/.local installiert.

1. Installieren Sie die Build-Voraussetzungen.

```
. /etc/os-release
test "$ID" = "debian" && test "$VERSION_ID" = "13"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-
dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils
zlibg-dev
```

2. Bauen Sie das geprüfte CPython 3.14.6.

```
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s  %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
```

3. Checken Sie BPM aus.

```
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

4. Installieren und migrieren Sie BPM.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

```
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

5. Starten Sie BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

6. Prüfen Sie das Ergebnis.

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

7. Brechen Sie sicher ab.

Bewahren Sie Prüfsumme und Protokolle auf; führen Sie keine Datenbankherabstufung aus und entfernen Sie Python erst nach der Diagnose.

Python 3.14.6, BPM, Migration, Dokumentation und Endpunkte sind vorbereitet.

Die Befehlsfolge wurde in einem sauberen Debian-13.5-Userspace-Container erfolgreich ausgeführt. CPython meldete die optionalen Module `_dbm` und `_zstd` als nicht verfügbar; der vollständige BPM-Ablauf war dennoch erfolgreich. Damit sind nativer Systemstart, Kernel, systemd, Desktop, Hardware, Host-Firewall und Produktivbetrieb nicht geprüft.

BPM aus dem Quellcode unter Fedora Linux 44 installieren

Vollständige BPM-0.9.1-Installation mit dem Python-3.14-Laufzeitsystem der Distribution.

Erforderlich sind Fedora 44, sudo, HTTPS und eine freigegebene BPM-Referenz.

Dies ist ein Einzelknoten-Quellbetrieb, kein systemd- oder Produktionsrezept.

1. Installieren Sie die Voraussetzungen.

```
. /etc/os-release
test "$ID" = "fedora" && test "$VERSION_ID" = "44"
sudo dnf install -y ca-certificates curl gcc gcc-c++ git make python3 python3-devel
export BPM_PYTHON=/usr/bin/python3
"$BPM_PYTHON" --version
```

2. Checken Sie BPM aus.

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. Installieren und migrieren Sie BPM.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. Starten Sie BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. Prüfen Sie das Ergebnis.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Brechen Sie sicher ab.

Bewahren Sie DNF-, pip-, Migrations-, Build- und Protokollausgaben auf und stufen Sie die Datenbank nicht herab.

BPM ist migriert, die Dokumentation gebaut und die Endpunkte antworten.

Die Befehlsfolge wurde in einem sauberen Fedora-44-Userspace-Container erfolgreich ausgeführt. Damit sind nativer Systemstart, Kernel, systemd, Desktop, Hardware, Host-Firewall und Produktivbetrieb nicht geprüft.

BPM von der Quelle unter Linux Mint 22.3 installieren

Führen Sie die vollständige Quellinstallationssequenz von BPM 0.9.1 unter Linux Mint 22.3 aus, indem Sie unter dem aktuellen Benutzer ein mit Prüfsummen verifiziertes CPython 3.14.6 installieren.

Verwenden Sie Linux Mint 22.3 (Zena) mit Sudo-Zugriff, ausgehendem HTTPS, mindestens 4 GB freiem Build-Space und einer genehmigten BPM 0.9.1 Git-Referenz.

Linux Mint 22.3 verwendet die Ubuntu-Noble-Paketbasis, deren Standard-Python unter den 3.14-Anforderungen von BPM liegt. Das System Python wird nicht ersetzt.

1. Überprüfen Sie Mint und installieren Sie die Build-Voraussetzungen für CPython.

```
. /etc/os-release
test "$ID" = "linuxmint" && test "$VERSION_ID" = "22.3"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-
dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils
zlib1g-dev
```

2. Erstellen Sie die verifizierte Quellversion CPython 3.14.6.

```
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
```

3. Sehen Sie sich die genehmigte BPM-Quellenreferenz an.

```
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

4. Erstellen Sie die Umgebung, installieren Sie BPM und migrieren Sie.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

5. BPM im Vordergrund starten.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

6. Überprüfen Sie Sonden und UI von einem zweiten Terminal aus.

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

7. Wenden Sie die Stopp- und Rollback-Grenze an.

Drücken Sie bei einem Fehler `Ctrl+C`, bewahren Sie Protokolle und Prüfsummenausgabe auf und führen Sie `alembic downgrade` nicht aus. Behalten Sie das benutzereigene Python-Präfix bei und checken Sie es bis zur Diagnose aus.

Der Linux Mint 22.3-Host verfügt über eine benutzereigene Python 3.14.6-Laufzeitumgebung, eine migrierte BPM-Quellenprüfung, eine erstellte Dokumentation, das Übergeben von Prüfpunkten, eine Antwort auf UI und Vordergrundprotokolle.

Die Befehlsfolge wurde in einem sauberen Linux-Mint-22.3-Userspace-Container erfolgreich ausgeführt. CPython meldete die optionalen Module `_dbm` und `_zstd` als nicht verfügbar; der vollständige BPM-Ablauf war dennoch erfolgreich. Damit sind nativer Systemstart, Kernel, systemd, Desktop, Hardware, Host-Firewall und Produktivbetrieb nicht geprüft.

BPM aus dem Quellcode im stabilen Manjaro-Zweig installieren

Vollständige Installation nach einer vollständigen Aktualisierung des stabilen Zweigs und Prüfung von Python 3.14.

Erforderlich sind der stabile Zweig ab Aktualisierung 2026-06-26, sudo, HTTPS und eine freigegebene BPM-Referenz.

Manjaro ist fortlaufend; Zweig und vollständig aktualisierter Paketstand müssen protokolliert werden.

1. Aktualisieren Sie das System und installieren Sie Voraussetzungen.

```
. /etc/os-release
test "$ID" = "manjaro"
test "$(pacman-mirrors -G)" = "stable"
sudo pacman -Syu --needed --noconfirm base-devel ca-certificates curl git python python-pip
pacman-mirrors -G
uname -r
export BPM_PYTHON=/usr/bin/python
"$BPM_PYTHON" --version
```

2. Checken Sie BPM aus.

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. Installieren und migrieren Sie BPM.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. Starten Sie BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
```

```
export BPM_RELOAD="true"
make dev
```

5. Prüfen Sie das Ergebnis.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Brechen Sie sicher ab.

Bewahren Sie Zweig-, Paket- und Protokolldaten auf; führen Sie keine partielle Paket- oder Datenbankherabstufung aus.

BPM und Dokumentation sind eingerichtet und die Endpunkte antworten.

Die Befehlsfolge wurde nach Protokollierung des Zweigs und des vollständigen Paketstands in einem sauberen Userspace-Container des stabilen Manjaro-Zweigs erfolgreich ausgeführt. Damit sind nativer Systemstart, Kernel, systemd, Desktop, Hardware, Host-Firewall und Produktivbetrieb nicht geprüft.

BPM aus dem Quellcode mit Windows WSL bereitstellen

Windows 10/11 für die WSL-Quellbereitstellung vorbereiten

Aktivieren Sie WSL, wählen Sie eine unterstützte Linux-Distributionsannahme und bestätigen Sie, dass BPM unter Linux und nicht als nativer Windows-Dienst ausgeführt wird.

Verwenden Sie Windows 10 oder Windows 11 mit der Berechtigung, WSL 2 zu aktivieren, installieren Sie eine Linux-Distribution wie Ubuntu LTS und führen Sie Befehle sowohl in PowerShell als auch in der WSL-Shell aus.

Dieses Betriebsanleitung dokumentiert die BPM-Quellbereitstellung über WSL. Der unterstützte Bewerbungsprozess ist weiterhin der Linux-Quellworkflow innerhalb der WSL; Native Windows-Installationsprogramme, Windows-Dienste, MSI-Pakete, geplante Aufgaben, IIS-Hosting und die Verwaltung von Windows-Produktionsdiensten werden in dieser Version nicht bereitgestellt.

1. Bestätigen Sie die WSL-Verfügbarkeit von PowerShell.

```
wsl --status  
wsl --list --verbose
```

Erwartetes Ergebnis: WSL ist installiert, WSL 2 ist verfügbar und die ausgewählte Linux-Distribution ist sichtbar.

2. Installieren Sie WSL und eine genehmigte Linux-Distribution, falls diese fehlen.

```
wsl --install  
wsl --install -d Ubuntu
```

Verwenden Sie Ihren von der Organisation genehmigten Vertriebsnamen. Starten Sie neu, wenn Windows dies verlangt, und erstellen Sie dann den Linux-Benutzer, wenn die Distribution zum ersten Mal startet.

3. Zeichnen Sie die Verteilung und die Dateisystemgrenze auf.

Platzieren Sie BPM unter dem WSL-Linux-Dateisystem, zum Beispiel `/home/<linux-user>/browser-policy-manager`. Vermeiden Sie es, den Checkout über `/mnt/c` auszuführen, da die Semantik des Windows-Dateisystems und die Virenprüfung dazu führen können, dass Python-Umgebungen, Dateiüberwachung und Berechtigungen unzuverlässig werden.

4. Bestätigen Sie, was dieser Windows-Pfad noch nicht unterstützt.

WARNUNG:

Dies ist keine native Windows-Installation. Das Betriebsanleitung bietet keine MSI/EXE-Paketierung, keinen Windows-Dienst, kein Dienstkonommodell, keine Ereignisanzeigeintegration, keine Windows-Firewallrichtlinie, kein Reverse-Proxy/TLS-Rezept, keine HA-Topologie, keine verwalteten Geheimnisse, keine Sicherungsautomatisierung oder keine Produktionshärtung.

Windows und WSL sind bereit für eine Linux-basierte BPM-Quellenbereitstellung mit klaren Plattform- und Dateisystemgrenzen.

Fahren Sie fort, indem Sie die WSL-Distributions-Shell öffnen und den Quell-Checkout im Linux-Home-Verzeichnis einrichten.

Das Auschecken der BPM-Quelle innerhalb der WSL einrichten

Installieren Sie die Linux-Voraussetzungen in WSL, klonen Sie BPM in das Linux-Dateisystem, erstellen Sie die virtuelle Umgebung, installieren Sie Abhängigkeiten und wenden Sie Migrationen an.

Eine WSL 2-Linux-Distribution wird installiert und geöffnet, und der Administrator verfügt über die genehmigte BPM-Repository-URL und den 0.9.1-Zweig oder das Tag.

Alle Befehle in diesem Thema werden in der WSL-Linux-Shell ausgeführt, es sei denn, der Schritt sagt ausdrücklich PowerShell. Dadurch wird sichergestellt, dass das Setup mit dem Betriebsanleitung der Linux-Quellbereitstellung übereinstimmt und Mehrdeutigkeiten beim nativen Windows-Pfad vermieden werden.

1. Installieren Sie die Linux-Voraussetzungen in der WSL-Distribution.

```
sudo apt-get update
sudo apt-get install -y git make curl python3 python3-venv python3-pip
python3 --version
```

Verwenden Sie den Paketmanager und die Paketnamen, die für Ihre ausgewählte WSL-Distribution zugelassen sind.

2. Klonen Sie BPM in das Linux-Home-Dateisystem.

```
cd ~
git clone <repository-url> browser-policy-manager
cd browser-policy-manager
git checkout <approved-0.9.1-ref>
```

Platzieren Sie den funktionierenden Checkout nicht unter /mnt/c für den unterstützten WSL-Quellworkflow.

3. Erstellen und aktivieren Sie die lokale Python-Umgebung.

```
python -m venv .env
source .env/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

Wenn die Distribution Python nur als python3 verfügbar macht, erstellen Sie die Umgebung mit `python3 -m venv .env` und verwenden Sie dann die aktivierte Umgebung.

4. Wenden Sie Datenbankmigrationen innerhalb des WSL-Checkouts an.

```
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

WARNUNG:

Die Datei SQLite ist eine Linux-Datei innerhalb der WSL. Bearbeiten oder verschieben Sie data/bpm.db nicht aus Windows-Tools, während BPM ausgeführt wird.

Der WSL-Checkout verfügt über Linux-Voraussetzungen, eine Repository-lokale virtuelle Umgebung, installierte Abhängigkeiten und eine migrierte lokale Datenbank.

Konfigurieren Sie Laufzeitvariablen und Browserzugriff, bevor Sie BPM starten.

WSL-Laufzeit und Browserzugriff konfigurieren

Legen Sie BPM-Laufzeitvariablen innerhalb der WSL fest, wählen Sie eine sichere Bindungsadresse, starten Sie den Quellprozess und identifizieren Sie die Browser-URL von Windows.

Der BPM-Checkout, die virtuelle Umgebung, Abhängigkeiten und Migrationen sind im WSL-Linux-Dateisystem bereit.

BPM liest BPM_ Umgebungsvariablen aus dem Linux-Prozess. Die moderne WSL leitet localhost normalerweise an Windows weiter, aber die WSL IP kann überprüft werden, wenn der Browserzugriff oder das Firewall-Verhalten unklar ist.

1. Legen Sie lokale Laufzeitvariablen in der WSL-Shell fest.

```
source .env/bin/activate
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
export BPM_HOST="0.0.0.0"
export BPM_PORT="8000"
export BPM_RELOAD="true"
```

BPM_HOST="0.0.0.0" ermöglicht dem Windows-Browser, den WSL-Prozess über die Localhost-Weiterleitung zu erreichen. Verwenden Sie 127.0.0.1 nur, wenn sich der Browser auch innerhalb der WSL befindet oder die Weiterleitung überprüft wurde.

2. Starten Sie BPM über den WSL-Checkout.

```
make dev
```

Das Makefile startet `uvicorn app.main:app --reload --port 8000`. Halten Sie das WSL-Terminal für Startmeldungen, Tracebacks und Datei-Watcher-Ausgaben geöffnet.

3. Identifizieren Sie die Windows-Browser-URL.

```
hostname -I
```

Probieren Sie zuerst `http://localhost:8000/profiles` unter Windows aus. Wenn die Localhost-Weiterleitung nicht verfügbar ist, verwenden Sie die erste von `hostname -I` gemeldete WSL IP, zum Beispiel `http://<wsl-ip>:8000/profiles`.

4. Halten Sie die Sicherheitsgrenzen explizit.

WARNUNG:

Das Öffnen des WSL-Ports für andere Hosts, das Ändern der Windows-Firewallregeln, das Hinzufügen von TLS oder das Platzieren eines Reverse-Proxys vor BPM liegt außerhalb dieses Themas zur WSL-Quellbereitstellung und muss auf Anleitungen zur Produktionsbereitschaft warten.

Der BPM-Quellprozess wird innerhalb der WSL ausgeführt und verfügt über einen identifizierten Windows-Browser-Zugriffspfad.

Überprüfen Sie Integrität, Bereitschaft, UI-Zugriff und WSL-spezifische Fehlermodi.

Die Bereitstellung der BPM-Quelle über WSL prüfen

Überprüfen Sie Sonden von WSL und Windows, bestätigen Sie die Route UI, überprüfen Sie Protokolle und beheben Sie häufige WSL-Netzwerk- und Dateisystemprobleme.

BPM wird vom WSL-Checkout mit den vorgesehenen Einstellungen für BPM_DATABASE_URL, Host, Port und Dokumentationsseite ausgeführt.

Die Überprüfung muss sowohl die Linux-Prozessantwort als auch den Windows-Browser-Zugriffspfad nachweisen. Es zertifiziert keine native Windows-Installation, Produktionshärtung, Reverse-Proxy-Verhalten, Backup-Bereitschaft oder Firefox-Laufzeitdurchsetzung.

1. Überprüfen Sie den Zustand und die Bereitschaft innerhalb der WSL.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Die erwarteten Antworten sind {"status": "ok"} und {"status": "ready", "ready": true}.

2. Überprüfen Sie die gleichen Sonden von Windows PowerShell.

```
Invoke-WebRequest http://localhost:8000/health
Invoke-WebRequest http://localhost:8000/health/ready
```

Wenn localhost nicht funktioniert, versuchen Sie es erneut mit der WSL IP von hostname -I. Notieren Sie, welche Adresse erfolgreich war.

3. Öffnen Sie die Route UI im Windows-Browser.

Verwenden Sie http://localhost:8000/profiles oder http://<wsl-ip>:8000/profiles. Die Bibliotheksroute sollte die BPM-Schnittstelle oder einen absichtlichen Leer-/Fehlerzustand laden, kein Traceback.

4. Beheben Sie WSL-spezifische Fehler, bevor Sie den BPM-Code ändern.

Bestätigen Sie bei Verbindungsfehlern, dass make dev noch ausgeführt wird, überprüfen Sie BPM_HOST, führen Sie wsl --list --verbose aus und aktualisieren Sie die WSL IP nach dem Neustart der Verteilung. Bei langsamer Dateiüberwachung oder Berechtigungsfehlern verschieben Sie den Checkout von /mnt/c nach /home/<linux-user>. Führen Sie erst aus, wenn der Umgebungspfad und die Netzwerkroute stabil sind.

WARNUNG:

Das Bestehen von Integritätsprüfungen über WSL beweist nicht die Produktionsbereitschaft, das Windows-Dienstverhalten, verwaltete Sicherungen oder die HA-Unterstützung.

Der Administrator hat Beweise dafür, dass BPM innerhalb der WSL startet, auf Tests reagiert, von Windows aus erreichbar ist und über keine unmittelbaren WSL-Netzwerk- oder Dateisystemblocker

verfügt.

Halten Sie die Ausgabe von PowerShell, WSL-Shell-Befehle, Variablennamen von BPM_*, Testantworten und Vordergrundprotokolle zur Fehlerbehebung zusammen, ohne Geheimnisse zu kopieren.

Eine BPM-Quellbereitstellung betreiben und aktualisieren

DevOps-Konfigurationsquellen prüfen

Identifizieren Sie die aktuellen BPM-Konfigurationsquellen, zeichnen Sie effektive Laufzeitwerte auf und trennen Sie normale Einstellungen von nicht unterstützten Erwartungen an die Geheimnisverwaltung.

BPM wird von der Quelle unter Linux oder über WSL bereitgestellt, und der Administrator kann die Shell-Umgebung, die optionale Datei `.env` und das Auschecken der Quelle überprüfen.

BPM lädt Einstellungen aus Umgebungsvariablen mit dem Präfix `BPM_` und aus einer Repository-Stammdatei `.env`. Das Konfigurationsmodell befindet sich in die dokumentierten Standardwerte. Diese Version definiert keinen Anwendungsgeheimnisspeicher, keinen Anmeldeinformations-Tresor, keine API-Token-Einstellung, keine Verschlüsselungsschlüsseleinstellung oder kein Produktionskonfigurationsprofil.

1. Listen Sie die für diesen Lauf verwendeten Konfigurationsquellen auf.

Zeichnen Sie auf, ob Werte aus Shell-Exporten, einem lokalen `.env` oder Standardwerten in die dokumentierten Standardwerte stammen. Behalten Sie `.env` lokal beim Checkout bei und schreiben Sie es nicht fest.

2. Zeichnen Sie die Laufzeiteinstellungen auf, die sich auf das Prozessverhalten auswirken.

Einschließlich `BPM_APP_NAME`, `BPM_APP_VERSION`, `BPM_DEBUG`, `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_API_PREFIX`, `BPM_DEFAULT_LOCALE`, `BPM_SUPPORTED_LOCALES`, `BPM_I18N_DIR` und `BPM_DOCUMENTATION_SITE_DIR`. Bei den Standardwerten handelt es sich um lokale Quellstandards, nicht um Produktionsrichtlinien.

3. Daten, Schema und netzwerkbezogene Einstellungen separat aufzeichnen.

Verfolgen Sie `BPM_DATABASE_URL`, `BPM_DB_ECHO`, `BPM_ENABLE_CORS`, `BPM_CORS_ALLOW_ORIGINS`, `BPM_SCHEMA_BASE_URL`, `BPM_SCHEMA_CACHE_DIR` und `BPM_SCHEMA_HTTP_TIMEOUT`. Behandeln Sie die Datenbank URL-Adressen und die Netzwerkursprünge als Betriebsnachweis, auch wenn sie kein Passwort enthalten.

4. Halten Sie Geheimnisse aus BPM-Konfigurationsbeispielen fern.

WARNUNG:

Die BPM-Quellenbereitstellungsdokumentation bietet keine Authentifizierungseinrichtung, verwaltete Geheimnisse, Konfiguration der Verschlüsselung im Ruhezustand, Tokenrotation, Mandantenisolation oder Geheimredaktionsgarantien. Wenn Ihre Umgebung diese Kontrollen benötigt, erfassen Sie sie als externe Plattformkontrollen und nicht als BPM-unterstützte Funktionen.

Die Bereitstellung verfügt über ein überprüftes Konfigurationsinventar, das mit den aktuellen Quelleinstellungen übereinstimmt und kein nicht unterstütztes Secret-Management-Verhalten impliziert.

Verwenden Sie die Themen Speicher, Netzwerk und Betriebsgrenzen, um die Auswirkungen der aufgezeichneten Werte zu überprüfen, bevor Sie BPM außerhalb einer kontrollierten lokalen Umgebung verfügbar machen.

Speicherung, Protokolle und Sicherungsnachweise planen

Suchen Sie die Quell-Bereitstellungsdatenbank, den Schema-Cache, die Dokumentationsseite, Vordergrundprotokolle und manuelle Sicherungsnachweise, ohne Anspruch auf verwaltete Sicherungs- oder Wiederherstellungsunterstützung zu erheben.

Das Bereitstellungsconfigurationsinventar umfasst `BPM_DATABASE_URL`, `BPM_SCHEMA_CACHE_DIR` und `BPM_DOCUMENTATION_SITE_DIR`.

Die Standardquellenbereitstellung speichert Anwendungsdaten in SQLite bei `sqlite+aiosqlite:///./data/bpm.db`, entsprechend `data/bpm.db` im Repository-Stammverzeichnis. Schema-Cache-Dateien befinden sich standardmäßig unter `app/schemas/mozilla`, und ein installiertes Dokumentationsartefakt wird von `app/documentation/site` gelesen, sofern vorhanden.

1. Identifizieren Sie die Dateien und Verzeichnisse, die den Betriebsstatus haben.

Zeichnen Sie den Repository-Pfad auf: `data/`, `data/bpm.db`, `app/schemas/mozilla` und `app/documentation/site`. Behalten Sie diese Pfade in WSL im Linux-Dateisystem und nicht unter `/mnt/c`.

2. Definieren Sie vor riskanten Arbeiten einen manuellen Beweissicherungsschritt.

Stoppen Sie den BPM-Prozess im Vordergrund, kopieren Sie `data/bpm.db` an einen kontrollierten Sicherungsort und zeichnen Sie Zeitstempel, Quellrevision, Datenbank-URL, Dateigröße und Prüfsumme auf. BPM bietet keine automatisierte Sicherung, keine Point-in-Time-Wiederherstellung oder getestete Wiederherstellungsorchestrierung.

3. Protokolle vom aktuellen Quellprozess erfassen.

Quellläufe schreiben Startmeldungen, Uvicorn-Zugriffsausgaben, Tracebacks, Migrationsfehler, Portkonflikte und Dokumentationsseitenwarnungen in das Vordergrundterminal, `stdout` oder `stderr`. In dieser Version sind kein dediziertes BPM-Anwendungsprotokollverzeichnis, keine Protokollrotationsrichtlinie, keine Anforderung ID, kein Prüfprotokoll oder keine Ereignisanzeigeintegration dokumentiert.

4. Kombinieren Sie Dateisicherungsnachweise mit Exportnachweisen, wenn die Profilübergabe wichtig ist.

Verwenden Sie vorhandene API-Exportbeispiele, um relevante `Firefox-policies.json`-Dokumente zu exportieren und den Antwortstatus, das Profil ID, die Revision, den Schemakanal und den Hash des exportierten Dokuments zu speichern. Behandeln Sie Exportnachweise nicht als vollständige Datenbankwiederherstellung.

Der Administrator weiß, wo sich der aktuelle Status der Quellbereitstellung befindet und welche manuellen Beweise vor Aktualisierungen oder Fehlerbehebungen gesammelt werden können.

Bevor Sie Quellrevisionen oder Datenbankschemata ändern, verwenden Sie das zukünftige Update-aus-Quelle-Betriebsanleitung und stoppen Sie, wenn die Sicherungs- oder Wiederherstellungsnachweise unvollständig sind.

Netzwerk, CORS und Sicherheitsbeschränkungen prüfen

Überprüfen Sie Host, Port, Neuladen, Integritätstests, CORS-StandardEinstellungen und Sicherheitsbeschränkungen, bevor Sie eine von der Quelle ausgeführte BPM-Instanz erreichbar machen.

Der Quellprozess kann mit `make dev` gestartet werden und der Administrator kann `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_ENABLE_CORS` und `BPM_CORS_ALLOW_ORIGINS` ändern.

Die aktuellen Standardeinstellungen sind für den lokalen Quellbetrieb optimiert: `BPM_HOST="0.0.0.0"`, `BPM_PORT="8000"`, `BPM_RELOAD="true"`, `BPM_ENABLE_CORS="true"` und zulässige Ursprünge entsprechend `["*"]`. Bei diesen Standardeinstellungen handelt es sich nicht um eine gehärtete Netzwerk-Offenlegungsrichtlinie.

1. Wählen Sie die engste Bindungsadresse für die aktuelle Umgebung.

Verwenden Sie `127.0.0.1` für lokale Linux-Prüfungen. Verwenden Sie `0.0.0.0` nur, wenn die WSL-Browserweiterleitung oder ein kontrolliertes Labornetzwerk dies erfordert. Notieren Sie den Grund für jede Nicht-Loopback-Bindung.

2. Gate-Betriebszugriff mit Integritäts- und Bereitschaftsprüfungen.

Verwenden Sie `GET /health` und `GET /health/ready` vor API-Beispielen oder UI Smoke Checks. Erfolgreiche Tests zeigen nur die Aktivität/Bereitschaft des Prozesses; Sie beweisen nicht den Datenbanksicherungsstatus, die Firefox-Durchsetzung, die Authentifizierung, die Autorisierung oder die Produktionstopologie.

3. Beschränken Sie CORS für jede gemeinsame Bewertung.

Für lokale Experimente kann ein zulässiges `BPM_CORS_ALLOW_ORIGINS` akzeptabel sein. Legen Sie für einen gemeinsam genutzten Host explizite Ursprünge fest und erfassen Sie, wer der Browserursprung ist. In der BPM-Dokumentation werden keine CORS-, CSRF-Schutzmaßnahmen mit Anmeldeinformationen, API-Authentifizierung oder Mandantenisolation definiert.

4. Halten Sie nicht unterstützte Sicherheitskontrollen vom BPM-Anspruch fern.

WARNUNG:

Diese Version bietet keine TLS-Beendigung, Reverse-Proxy-Rezepte, Firewall-Richtlinie, Ratenbegrenzung, Anforderungssignierung, Audit-Protokollierung, SSO, OAuth/OIDC-Setup, rollenbasierte Zugriffskontrolle, Schwachstellenscans oder HA-Failover. Dokumentieren Sie diese als externe Kontrollen, wenn Ihre Umgebung sie hinzufügt.

Der von der Quelle ausgeführte BPM-Netzwerkstatus wird als kontrollierte lokale oder Laborexposition und nicht als Produktionssicherheitsarchitektur dokumentiert.

Wenn Tests fehlschlagen oder die Gefährdung unklar ist, verwenden Sie die Themen zur Überprüfung der Quellbereitstellung, bevor Sie mit den API-Integrationsbeispielen fortfahren.

Betriebsgrenzen einer Quellbereitstellung dokumentieren

Erstellen Sie eine Betriebsübergabenotiz, die den Laufzeitbenutzer, das Prozessmodell, unterstützte Prüfungen und verzögerte Produktionsfunktionen angibt.

Die Konfigurations-, Speicher-, Backup-, Netzwerk- und CORS-Überprüfungen wurden für die von der Quelle ausgeführte Instanz abgeschlossen.

Die BPM Administrator/DevOps-Dokumentation unterstützt derzeit den Source-Run-Betrieb für eine kontrollierte Auswertung. Es bietet noch nicht Verteilungsinstallationsprogramme, native Dienste, offizielle Daemonisierung, Produktionstopologie oder vollständige Betriebs-Betriebsanleitungen für jede Unternehmenssteuerung.

1. Zeichnen Sie den Laufzeitbenutzer und das Prozessmodell auf.

Identifizieren Sie den Shell-Benutzer, der Eigentümer des Checkout-Prozesses ist, `.venv`, `data/` und den laufenden `make dev-` oder `uvicorn app.main:app --reload --port 8000`-Prozess. Beanspruchen Sie kein unterstütztes Dienstkonto, keine Systemd-Einheit, keinen Windows-Dienst oder keine Neustartrichtlinie, es sei denn, Ihre externe Plattform stellt diese bereit und besitzt sie.

2. Erfassen Sie die Prüfungen, die heute unterstützt werden.

Einschließlich Quellrevision, aktive Konfigurationswerte, Migrationsstatus, Routenrauch `/health`, `/health/ready`, UI, API-Beispielstatus, Überprüfung des Vordergrundprotokolls und manuelle Sicherungsnachweise. Verknüpfen Sie API-Beweise mit wiederverwendbaren API-Beispielen, anstatt Endpunktprozeduren zu duplizieren.

3. Erfassen Sie zurückgestellte Fähigkeiten als Blocker oder externe Verantwortlichkeiten.

Rufen Sie gepackte Installationsprogramme, native Linux- oder Windows-Dienste, Reverse-Proxy- und TLS-Rezepte, HA-Clustering, verwaltete Geheimnisse, Authentifizierungskonfiguration, Sicherungsautomatisierung, Wiederherstellungsorchestrierung, Überwachungs-Dashboards, Protokollrotation und Produktionshärtung auf, wie in der BPM-Quelldokumentation nicht bereitgestellt.

4. Hängen Sie benutzerseitige Wiederherstellungsreferenzen an, ohne den Besitz zu verschieben.

Wenn ein Bediener die UI-Symptome verstehen muss, verweisen Sie auf benutzerbezogene Fehlerbehebungsmaßnahmen wie die Wiederherstellung der Produktverbindung. Behalten Sie die Bereitstellung, API, Sicherung und Netzwerkeigentümerschaft im Administrator-/DevOps-Handbuch bei.

In der Übergabenotiz wird angegeben, was die aktuelle Quellbereitstellung unterstützen kann, welche Beweise dies belegen und welche Produktionsverantwortlichkeiten außerhalb von BPM verbleiben.

Verwenden Sie diese Randnotiz als Ausgangspunkt für das Update-from-Source-Betriebsanleitung und spätere Produktions-/HA/Reverse-Proxy-Dokumentationsaufgaben.

Nachweise vor einer Quellaktualisierung vorbereiten

Sammeln Sie die Quellrevision, Konfiguration, Datenbanksicherung, Profilexportnachweise und Stoppbedingungen, bevor Sie BPM auf eine neuere Quellrevision verschieben.

BPM wird von einem Linux- oder WSL-Quellen-Checkout ausgeführt und der Administrator hat die Konfigurations-, Speicher-, Sicherungs- und Betriebsgrenzenüberprüfungen abgeschlossen.

Eine Update-from-Source-Prozedur ändert Code, Abhängigkeiten, Datenbankschema und Dokumentationsausgabeerwartungen. BPM bietet kein automatisches Rollback, Datenbank-Downgrade-Garantien, verwaltetes Backup oder Paket-Upgrade-Tools. Behandeln Sie die Beweissammlung als ein erforderliches Pre-Update-Gate.

1. Stoppen Sie den BPM-Prozess im Vordergrund und zeichnen Sie den aktuellen Quellstatus auf.

```
git status --short  
git rev-parse --short HEAD
```

Fahren Sie nur fort, wenn der Arbeitsbaumstatus verstanden und die aktuelle Quellrevision als Rollback-Referenz aufgezeichnet ist.

2. Zeichnen Sie effektive Konfigurations- und Laufzeitpfade auf.

Erfassen Sie die aktiven Werte `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR`, `BPM_SCHEMA_CACHE_DIR`, `BPM_HOST`, `BPM_PORT` und ob Werte aus Shell-Exporten oder `.env` stammen. Kopieren Sie keine Geheimnisse in die Update-Notiz.

3. Erstellen Sie manuelle Datenbanksicherungsnachweise, während BPM gestoppt ist.

```
mkdir -p backups  
cp data/bpm.db backups/bpm-pre-update.db  
sha256sum backups/bpm-pre-update.db
```

Zeichnen Sie Zeitstempel, aktuelle Revision, Datenbank-URL, Dateigröße und Prüfsumme auf. Wenn die Bereitstellung nicht `data/bpm.db` verwendet, sichern Sie stattdessen den von `BPM_DATABASE_URL` aufgelösten Pfad.

4. Exportieren Sie kritische Profilnachweise, bevor Sie den Code ändern.

Verwenden Sie die vorhandenen API-Exportbeispiele, um erforderliche `Firefox-policies.json`-Dokumente zu speichern und den Antwortstatus, das Profil ID, die Revision, den Schemakanal und den Hash des exportierten Dokuments aufzuzeichnen.

WARNUNG:

Wenn Datenbanksicherungs- oder Exportnachweise fehlen, stoppen Sie die Aktualisierung. Ersetzen Sie fehlende Sicherungsnachweise nicht durch die Annahme, dass Git-Rollback oder `alembic downgrade` sicher sind.

Das Update verfügt über einen verifizierten Beweissatz vor der Änderung und einen bekannten früheren Quellenverweis.

Fahren Sie mit der Aktualisierung der Quellrevision und der Abhängigkeiten erst fort, nachdem der Beweissatz vollständig und überprüft ist.

Quellrevision und Abhängigkeiten aktualisieren

Verschieben Sie das Auschecken auf eine genehmigte neuere Revision und installieren Sie die Python-Umgebung neu, ohne Abhängigkeits- oder Arbeitsbaumfehler auszublenden.

Der Beweissatz vor dem Update ist vorhanden, die vorherige Revision ist aufgezeichnet und ein genehmigter Zielzweig, Tag oder Commit ist bekannt.

BPM-Quellaktualisierungen sind Vorgänge in der Git- und Python-Umgebung, die im vorhandenen Checkout ausgeführt werden. Bei diesem Verfahren wird keine Paketdistribution, kein nativer Dienst oder kein Produktions-Upgrade-Manager installiert.

1. Rufen Sie die genehmigte Quellenreferenz ab.

```
git fetch --tags
git status --short
```

Stoppen, wenn lokale Änderungen unerwartet sind. Behalten Sie benutzereigene oder Website-eigene Änderungen bei, anstatt sie zu überschreiben.

2. Wechseln Sie zur genehmigten neueren Quellrevision.

```
git checkout <approved-new-ref>
git rev-parse --short HEAD
```

Zeichnen Sie die Zielrevision auf und vergleichen Sie sie mit dem genehmigten Änderungsticket oder Versionshinweis.

3. Aktualisieren Sie die Repository-lokale Python-Umgebung.

```
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

Verwenden Sie die vorhandene `.venv`, es sei denn, der Aktualisierungshinweis erfordert ausdrücklich eine Neuerstellung. Notieren Sie Abhängigkeitsfehler, bevor Sie die Datenbank ändern.

4. Stopp bei ungelösten Quell- oder Abhängigkeitsfehlern.

WARNUNG:

Wenn das Auschecken, die Abhängigkeitsinstallation oder die Aktivierung der virtuellen Umgebung fehlschlägt, führen Sie keine Migrationen aus. Kehren Sie nur dann zur aufgezeichneten vorherigen Revision zurück, wenn die Datenbank nicht geändert wurde, installieren Sie dann den vorherigen Abhängigkeitssatz neu und führen Sie die Smoke-Checks vor dem Update erneut aus.

Die Checkout- und Python-Umgebung befinden sich auf der genehmigten Zielrevision oder die Aktualisierung wurde vor Datenbankänderungen gestoppt.

Führen Sie Datenbankmigrationen und Dokumentationsprüfungen erst aus, nachdem die Abhängigkeitsaktualisierung erfolgreich war.

Migrationen und Prüfungen der Quellaktualisierung durchführen

Wenden Sie Datenbankmigrationen an und prüfen Sie die aktualisierte BPM-Quellbereitstellung nach einer Änderung der Quellrevision.

Das Auschecken und die Abhängigkeiten wurden erfolgreich aktualisiert und die Datenbanksicherungsnachweise wurden gesammelt, während BPM gestoppt wurde.

Aktualisierungen können Datenbankmigrationen enthalten. BPM dokumentiert die Vorwärtsmigration mit `alembic upgrade head`; ein automatisches Downgrade oder sicheres Schema-Rollback wird nicht zugesagt.

1. Datenbankmigrationen auf die vorgesehene Datenbank anwenden.

```
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

Verwenden Sie den tatsächlichen Bereitstellungswert für `BPM_DATABASE_URL`. Stoppen Sie sofort, wenn die Migrationsausgabe unklar ist oder fehlschlägt.

2. Starten Sie den aktualisierten BPM-Prozess und prüfen Sie seine aktuelle Oberfläche.

Prüfen Sie Aktivität und Bereitschaft und öffnen Sie anschließend die Bibliotheksroute. Falls Dokumentation für die Bereitstellung erforderlich ist, muss `/help/` ein bereits installiertes Artefakt statt eines fehlenden oder inkompatiblen Zustands melden.

3. Bei Migrations- oder Laufzeitprüfungsfehlern anhalten.

WARNUNG:

Gehen Sie nach einer Migration nicht davon aus, dass ein Git-Checkout-Rollback ausreicht. Führen Sie `alembic downgrade` nur aus, wenn ein geprüfter Versionshinweis die Unterstützung für das genaue Revisionspaar ausdrücklich bestätigt. Verwenden Sie bei einem erforderlichen Rollback die Datenbanksicherung vor dem Update als sichere Wiederherstellungsbasis.

Die aktualisierte Quellrevision hat Migrationen und Laufzeitprüfungen abgeschlossen, oder die Aktualisierung wurde an einem benannten Fehlerpunkt gestoppt.

Starten Sie BPM und führen Sie das Post-Update-Überprüfungsthema aus, bevor Sie die Instanz an Benutzer oder Integratoren zurückgeben.

Das Update prüfen und Rollback-Stopp-Bedingungen anwenden

Starten Sie den aktualisierten BPM-Prozess, überprüfen Sie Probes, UI, API-Beweise und den Dokumentationsstatus und entscheiden Sie dann, ob Sie ihn akzeptieren, stoppen oder aus der Sicherung wiederherstellen möchten.

Migrationen, Quellrauch und Überprüfungen zur Neuerstellung der Dokumentation wurden abgeschlossen oder es wurde ein Fehlerpunkt aufgezeichnet.

Die Überprüfung nach dem Update beweist, dass die aktualisierte Quellausführung für die aktuell kontrollierte Umgebung verwendbar ist. Es beweist nicht die Produktionsbereitschaft, das Reverse-Proxy-Verhalten, HA, die verwaltete Sicherung oder die Durchsetzung der Firefox-Laufzeit.

1. Starten Sie den aktualisierten Quellprozess und prüfen Sie die Probes.

```
make dev
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Die erwarteten Antworten bleiben {"status": "ok"} und {"status": "ready", "ready": true}.

2. Überprüfen Sie das Produkt UI, den API-Beweis und den Dokumentationsstatus.

Öffnen Sie `http://127.0.0.1:8000/profiles`, führen Sie die erforderlichen wiederverwendbaren API-Beispiele aus, bestätigen Sie bei Bedarf exportierte `policies.json`-Beweise und stellen Sie sicher, dass `/help/` keine veraltete, inkompatible oder fehlende Dokumentation für das installierte Artefakt meldet.

3. Akzeptieren Sie die Aktualisierung nur, wenn alle erforderlichen Überprüfungen aufgezeichnet wurden.

Erfassen Sie die vorherige Revision, die Zielrevision, das Migrationsergebnis, das Quell-Smoke-Ergebnis, das Dokumentationsvalidierungs-/Build-Ergebnis, die Zustands-/Bereitschaftsreaktionen, das UI-Smoke-Ergebnis, API-/Exportnachweise und alle bekannten verzögerten Produktionskontrollen.

4. Wenden Sie Rollback- oder Stoppbedingungen an, ohne eine automatische Wiederherstellung zu erfinden.

Wenn vor Migrationen ein Fehler auftritt, kehren Sie zur vorherigen Quellrevision zurück und installieren Sie die Abhängigkeiten neu. Wenn nach Migrationen ein Fehler auftritt, stoppen Sie BPM, stellen Sie die überprüfte Datenbanksicherung vor dem Update wieder her, kehren Sie zur vorherigen Quellrevision zurück, installieren Sie Abhängigkeiten neu und wiederholen Sie die Integritäts-/UI-Rauchprüfungen. Wenn Sicherungsnachweise fehlen, die Wiederherstellung nicht überprüft wurde oder sich Daten nach der Migration geändert haben, stoppen Sie den Vorgang und eskalieren Sie ihn, anstatt einen Rollback-Erfolg zu beanspruchen.

WARNUNG:

Markieren Sie das Update nicht als abgeschlossen, wenn Tests fehlschlagen, die Dokumentation veraltet ist, Exporte unerwartet abweichen, Migrationen unsicher sind oder Rollback-Beweise unvollständig sind.

Das Update wird entweder mit vollständigem Nachweis akzeptiert, bei einem bekannten Blocker gestoppt oder mit explizitem Nachweis aus der Sicherung vor dem Update wiederhergestellt.

Verwenden Sie die resultierenden Beweise in der Betriebsübergabenotiz und in der zukünftigen Produktions-/HA/Reverse-Proxy-Planung.

BPM-Zustand und -Bereitschaft prüfen

Verwenden Sie die aktuellen Funktions- und Bereitschaftsprüfungen als eine einfache Vorabprüfung des Betriebszustands vor API-Arbeitsabläufen.

`$BPM_BASE_URL` festlegen. Für Host-Setup, Dienststart, Protokolle, Aktualisierungen von der Quelle und Details zum Betriebssystem verwenden Sie das Administratorhandbuch zur Bereitstellung aus dem Quellcode. Dieses API-Thema dokumentiert nur die Integrationsprüfung.

Diese Aufgabe umfasst API-HEALTH-001 und API-HEALTH-002. BPM stellt präfixfreie Prüfpunkte bei `/health` und `/health/ready` bereit. BPM stellt über diese Sonden keine Abhängigkeitsdiagnose zur Verfügung; Sie sind nicht versioniert und ersetzen keine produktspezifischen Wiederholungs- oder Vorfalldrichtlinien.

1. Rufen Sie die Funktionsprüfung auf, bevor Sie eine kurze API-Sequenz starten.

Anforderungsbeispiel: `GET $BPM_BASE_URL/health`. Antwortbeispiel: `200 {"status": "ok"}`. Fehlerbeispiel: Derzeit ist kein Anwendungsfehler deklariert; Transport-, Gateway- oder Prozessfehler werden außerhalb von BPM behandelt.

2. Rufen Sie den Bereitschaftstest auf, bevor Vorgänge, die vom Abschluss des Anwendungsstarts abhängen, abgeschlossen werden.

Anforderungsbeispiel: `GET $BPM_BASE_URL/health/ready`. Antwortbeispiel: `200 {"status": "ready", "ready": true}`. Fehlerbeispiel: Derzeit ist kein Hauptteil für den herabgesetzten Zustand oder Abhängigkeitsdetails verfügbar.

3. Verwenden Sie die Bereitschaft als Tor, nicht als vollständiges Abhängigkeitsinventar.

Wenn `ready true` ist, kann das integrierende Produkt mit Ermittlungs-, Validierungs-, Import-, Export- oder Profillebenszyklusaufrufen fortfahren. Aus dieser Antwort dürfen keine Details zum Datenbankzustand, Details zum Schema-Cache, zum Status des Dokumentationsportals, zum Warteschlangenstatus oder zur Bereitstellungstopologie abgeleitet werden.

4. Führen Sie Abfragen durch und wiederholen Sie Entscheidungen im integrierenden Produkt.

BPM deklariert kein Abfrageintervall, keinen Retry-After-Header, kein Service-Level-Ziel, keinen Leistungsschaltervertrag, keine Anfrage ID oder keinen Webhook für Probe-Änderungen. Speichern Sie Ihren eigenen Zeitstempel, beobachteten Status, Zeitüberschreitung und Wiederholungsentscheidung.

5. Trennen Sie API-Handshake-Prüfungen von der Fehlerbehebung bei der Quellbereitstellung.

Wenn Probes nicht erreichbar sind oder die Antwort von den Beispielen abweicht, wechseln Sie zum Quell-Deployment-Administratorhandbuch für Prozess-, Umgebungs-, Datenbank-, Protokoll- und Update-from-Source-Prüfungen, anstatt diese API-Integrationsaufgabe in Bereitstellungsanweisungen zu erweitern.

6. Zeichnen Sie das operative Handshake-Ergebnis mit der nachgelagerten API-Aktion auf.

WARNUNG:

Eine erfolgreiche Integritäts- oder Bereitschaftsantwort authentifiziert keine Aufrufer, autorisiert keine Vorgänge, reserviert keine Profilrevisionen, garantiert keinen späteren API-Erfolg und beweist auch nicht die Durchsetzung der Firefox-Laufzeit. Behandeln Sie es als ein kleines Vorprüfungssignal.

Das integrierende Produkt verfügt über einen minimalen und aktuellen BPM-Betriebs-Handshake für Lebendigkeit und Einsatzbereitschaft, ohne von undokumentiertem Bereitstellungsverhalten abhängig zu sein.

Fahren Sie nach einer erfolgreichen Bereitschaftsprüfung mit der OpenAPI-Erkennung oder dem spezifischen API-Arbeitsablauf-Thema für Validierung, Import/Export oder Profillebenszyklusvorgänge fort.

BPM über die API integrieren

Integrationszielgruppe und API-Grenze

Der API-Integrationsabschnitt im Administrator-/DevOps-Handbuch richtet sich an externe Werkzeuge, die die aktuelle programmatische API von BPM verwenden, ohne nicht unterstützte Produktgarantien anzunehmen.

Zielgruppe

Verwenden Sie dieses Handbuch, wenn Sie ein Konfigurations-, Compliance-, Orchestrierungs-, Inventar- oder Kontrollprodukt erstellen oder überprüfen, das Profil- und Firefox-Richtliniendaten mit BPM austauscht. Die Hauptleser sind API-Integratoren, Sicherheitsprüfer, Release-Ingenieure und Betreuer, die einen wiederholbaren Vertrag für Profilsynchronisierung, Validierungs-Gates, Richtlinienimport/-export, Compliance-Metadaten, Inventar und Gesundheitsprüfungen benötigen.

Beweisgrenze

Der Abschnitt basiert auf dem generierten OpenAPI-Vertrag, dem gepflegten API-Inventar, der Routen-/Modellquelle und gezielten API-Tests. Er dokumentiert die API, die BPM derzeit bereitstellt. Er genehmigt keine neuen Endpunkte, Sicherheitsmodelle, Datenschemata, Wiederholungssemantiken oder Kompatibilitätsversprechen.

Unterstützte Verbraucherrollen

Unterstützte Verbraucher sind generische Rollen: Konfigurationsmanager, Compliance-Scanner, Richtlinienüberprüfungsgate, Profilinventursammler, Bereitstellungskoordinator und Betriebsmonitor. Sie können Dienste, Profile, Firefox-Import/Export, Validierung und Integritätsvorgänge aufrufen, die im API-Inventar aufgeführt sind.

Nicht unterstützte Ansprüche

Beschreiben Sie keinen produktspezifischen Connector, keine Marktplatzintegration, keine zertifizierte Integration, keine Partnerschaft, keinen verwalteten Agenten oder keinen Anbieter-Arbeitsablauf, es sei denn, dieser Connector wird tatsächlich ausgeliefert. HTML-Routen, statische Assets, generiertes Schema Benutzeroberflächen und Python-Helfer mit Unterstrich-Präfix sind keine stabile Maschinenintegration API-Schnittstellen.

Dateneigentum

BPM besitzt das normalisierte Profilmodell, den Validierungsstatus, die Revision, den Lebenszyklusstatus, das undurchsichtige Compliance-Objekt und das kanonische Firefox-Export-

Rendering. Integrierende Produkte besitzen ihre Planung, Autorisierung, Prüfprotokoll, Wiederholungsrichtlinie, Ausnahmeaufzeichnung, Bereitstellungsentscheidung und jegliche Laufzeitüberprüfung außerhalb von BPM.

Unterstützte Integrationsmuster

BPM unterstützt generische API-Muster für Profil-, Validierungs-, Firefox-Dokument-, Metadaten-, Inventar- und Gesundheitsworkflows.

Profilsynchronisierung

Ein integrierendes Produkt kann Profile auflisten, ein Profil nach ID lesen, ein normalisiertes Profil erstellen, veränderbare Profelfelder patchen, die Bibliothek archivieren, wiederherstellen, dauerhaft löschen oder zurücksetzen. Die Synchronisierung muss das Profil ID und die Revision als BPM-eigenen Status behandeln und darf keine Massenvorgänge, Namensmutationen oder automatische Rollbacks über mehrere Anforderungen hinweg annehmen.

Validierungs-Gates

Ein Validierungsgate kann ein kanonisches `policies.json`-Kandidatendokument an den unterstützten Validierungsendpunkt senden, bevor es von einem Bereitstellungsworkflow akzeptiert wird. Das Gate muss den Antworttext sowie den Statuscode prüfen, da die Validierung mit `ok=false` einen erfolgreichen HTTP-Status zurückgeben kann.

Richtlinienimport und -export

Eine Integration kann Firefox `policies.json` als BPM-Profil importieren und ein gespeichertes BPM-Profil zurück in das kanonische Firefox-JSON exportieren. Tauschen Sie das normalisierte Profilmodell nicht mit der kanonischen Firefox-Dokumentform aus; Import/Export ist die Grenze zwischen diesen Darstellungen.

Übergabe der Compliance-Metadaten

Die API stellt ein undurchsichtiges `compliance`-Objekt bei Profilerstellungs-, Aktualisierungs- und Importvorgängen bereit. Integrationen können ihre eigenen Metadaten über BPM weiterleiten, aber BPM veröffentlicht kein versioniertes Schema für dieses Objekt und validiert keine externe Compliance-Semantik.

Bestands- und Gesundheitsprüfungen

Verbraucher im Inventarstil können Profile auflisten, Statistiken für denselben Filterkontext anfordern, einzelne Profile lesen und Service-, Liveness- und Bereitschaftsendpunkte aufrufen. Integritätsantworten sind absichtlich kompakt und legen keine Abhängigkeitsdiagnostik, Dokumentationsstatus oder Bereitstellungsbereitschaft offen, die über die aktuellen festen Felder hinausgehen.

Nicht unterstützte Muster

Das Handbuch dokumentiert keine Authentifizierung, Autorisierung, Ratenbegrenzungen, Idempotenzschlüssel, Anforderung Kennungen, Webhooks, Massenexport, Massen-CRUD, Cursor-Paginierung, produktspezifische Konnektoren, Anbieterpartnerschaften oder Transaktionsorchestrierung, da BPM diese Garantien nicht offenlegt.

API-Konventionen

Verwenden Sie diese Konventionen, wenn Sie BPM OpenAPI-Verträge lesen und Integrationsanfragen erstellen.

Basis URL-Adressen und Erkennung

Beispiele verwenden `$BPM_BASE_URL` als Bereitstellungsplatzhalter. Entdecken Sie den aktuellen Vertrag von `/openapi.json` und, nur zur interaktiven Ansicht, das FastAPI-Schema Benutzeroberflächen unter `/docs` und `/redoc`. Das zukünftige Produktdokumentationsportal ist für `/help/` reserviert. Behandeln Sie daher `/docs` nicht als DITA-Dokumentationsroute.

Inhaltstypen

Die meisten programmgesteuerten Vorgänge verwenden JSON-Anfrage- und Antworttexte. Der Firefox-Import akzeptiert `application/json` und `multipart/form-data`; Nicht unterstützte Medientypen können 415 zurückgeben. Der Firefox-Export gibt `application/json` zurück und kann Content-Disposition hinzufügen, wenn eine Download-Ausgabe angefordert wird.

Bezeichner und Lebenszyklus

Profil Kennungen sind ganzzahlige Pfadkennungen im Besitz von BPM. Speichern Sie die zurückgegebenen Werte `id` und `revision` nach Erstellungs-, Import-, Lese- und Aktualisierungsvorgängen. Profilnamen sind eindeutig und können nicht über `ProfileUpdate` geändert werden. Lebenszykluswerte sind `aktiv`, `archiviert` und `alle`; `include_deleted` ist ein Kompatibilitätsflag, das zusammen mit dem Lebenszyklus interpretiert werden muss.

Modelle und Schemakanäle

`ProfileCreate`, `ProfileUpdate` und `ProfileRead` sind BPM-normalisierte Profilformen. Der Import/Export von Firefox verwendet das kanonische `policies.json` mit der obersten Ebene `policies`. Die Validierung verwendet `ValidationRequest` und unterstützte Schemakanäle wie Firefox ESR und Release; Bei einer einfachen Richtlinienzuordnung bleibt das Kompatibilitätsverhalten erhalten, nicht die bevorzugte äußere Form.

Filtern, Sortieren und Paginierung

Profillisten- und Statistikoperationen unterstützen Suche, Schema, Validierungsstatus, Lebenszyklus und Filter zum Einschließen und Löschen. Die Listenvorgabe ist mit dem Standardwert `limit=50`, dem Maximalwert 200 und `offset=0` versetzt/begrenzt. Beim Sortieren werden aktuell beschriebene Felder wie `created_at`, `updated_at`, `name`, `schema_version` und `id` in der Reihenfolge `asc` oder `desc` verwendet.

Status- und Fehlerbehandlung

Verwenden Sie Statuscodes und Körperform zusammen. 200, 201 und 204 sind Erfolgsfamilien, aber die Validierung kann 200 mit `ok=false` zurückgeben. Fehlertexte verwenden FastAPI `detail` und können eine Zeichenfolge, eine Validierungsliste oder ein BPM-Objekt mit `message`, `error` oder `issues` enthalten.

Aktuelle API-Einschränkungen und Sicherheitsbeschränkungen

BPM dokumentiert aktuelle Einschränkungen deutlich, sodass aus Integrationen keine Garantien abgeleitet werden können, die nicht offengelegt werden.

Sicherheitseinschränkungen

Die aktuelle API verfügt über keine Authentifizierungs- oder Autorisierungsschicht. CORS verwendet standardmäßig *-Ursprünge, es sei denn, die Bereitstellungseinstellungen überschreiben dies. Die Dokumentation darf keine API-Token-Authentifizierung, OAuth, rollenbasierte Zugriffskontrolle, Mandantenisolation oder Produktionsautorisierungsverhalten implizieren.

Versionierung und Kompatibilität

Die API hat kein Versionspräfix, keine Kompatibilitätsrichtlinie, keinen Deprecation-Header oder keine Medientypversion. Integrationen sollten die BPM-Version, mit der sie testen, anheften und `/openapi.json`, das API-Inventar und die Versionshinweise beobachten, anstatt eine garantierte Abwärtskompatibilität anzunehmen.

Parallelität, Idempotenz und Wiederholung

Nur PATCH unterstützt optimistische Parallelität durch optionales `expected_revision`. Beim Erstellen, Importieren, Löschen, Wiederherstellen, Zurücksetzen und Exportieren werden weder ETags noch Vorbedingungsheader, Idempotenzschlüssel oder Wiederholungsverträge angezeigt. Verbraucher müssen ihr eigenes Verhalten für sichere Wiederholungsversuche und die Wiederherstellung doppelter Namen entwerfen.

Massen- und Transaktionsgrenzen

BPM stellt keinen Massenprofil-CRUD-Endpunkt, keinen Massenexport-Endpunkt, keine Cursor-Paginierung und keinen Multi-Operations-Transaktionsvertrag zur Verfügung. Eine fehlgeschlagene mehrstufige Integration wird von BPM nicht automatisch zurückgesetzt; Das integrierende Produkt verfügt über Wiederherstellungssequenzierungs- und Prüfaufzeichnungen.

Einschränkungen bei Fehlerumschlägen

Fehlerantworten verwenden keinen stabil versionierten Umschlag. Die Status 400, 404, 409, 415, 422 und 503 können verschiedene `detail`-Formen haben. Verbraucher müssen nach Betrieb, Status und dokumentierter Körperform verzweigen.

Ausgeschlossene und HTML Oberflächen

Die sechs OpenAPI-web-Operationen sind HTML-Produktrouten, keine stabile Maschinenintegration API-Schnittstellen. `/i18n/{locale}.json`, `/favicon.ico`, `/static/*`, das generierte Schema Benutzeroberflächen und Python-Helfer mit Unterstrich-Präfix sind von Themen zu Integrationsvorgängen ausgeschlossen.

Grenzwerte für Gesundheitsreaktionen

`/health` und `/health/ready` stellen nur kompakte Statusfelder bereit. Readiness meldet keine Diagnose pro Abhängigkeit, Datenbankdetails, Schemazustand, Dokumentationsportalstatus oder Bereitstellungsgenehmigung. Betriebsmonitore sollten sie als minimale Handshake-Endpunkte behandeln.

DevOps-API-Beispiele und Umgebungsvorlagen wiederverwenden

Verwenden Sie sprachneutrale Anforderungsmuster, Umgebungsvorlagen, kopierbare curl-Befehle, mehrteilige Beispiele und kompakte Python-Assertionen für aktuelle BPM-API-Arbeitsabläufe.

Legen Sie `BPM_BASE_URL` für die zu testende BPM-Instanz fest. Fügen Sie Anmeldeinformationen, Mandantennamen, persönliche Hostnamen, private Hosts oder das Produktionsprofil Kennungen nicht in wiederverwendbaren Beispielen ein. BPM dokumentiert derzeit keine API-Authentifizierung.

Diese Aufgabe umfasst API-HEALTH-001, API-HEALTH-002, API-PROFILE-001, API-PROFILE-003, API-PROFILE-004, API-PROFILE-005, API-FF-001, API-VAL-001 und API-FF-002. In den Beispielen werden absichtlich `$BPM_BASE_URL`, synthetische Namen, `release-153`, Antwortzusicherungen und die verfügbare Vorrichtung Kennungen anstelle von umgebungsspezifischen Hosts oder Geheimnissen verwendet.

1. Erstellen Sie eine wiederverwendbare Umgebungsvorlage, bevor Sie curl-Anfragen kopieren.

Anforderungsbeispiel: `export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"`.

Antwortbeispiel: Die Shell behält eine Basis-URL, einen Schemakanal, einen synthetischen Profilnamen und einen optionalen Importpfad für die gesamte Ausführung bei. Fehlerbeispiel: Die Shell beendet die Zuweisung, wenn ein erforderlicher Wert fehlt.

```
# .env.devops-example.template
BPM_BASE_URL=
BPM_SCHEMA_CHANNEL=release-153
BPM_PROFILE_NAME=docs-api-example
BPM_JSON_IMPORT_PATH=./policies.json

export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"
export BPM_SCHEMA_CHANNEL="${BPM_SCHEMA_CHANNEL:-release-153}"
export BPM_PROFILE_NAME="${BPM_PROFILE_NAME:-docs-api-example}"
export BPM_JSON_IMPORT_PATH="${BPM_JSON_IMPORT_PATH:-./policies.json}"
```

2. Begrenzen Sie jede wiederverwendbare Sequenz mit Integritäts- und Bereitschaftsprüfungen.

Anforderungsbeispiel: `curl -fsS "$BPM_BASE_URL/health"` und `curl -fsS`

`"$BPM_BASE_URL/health/ready"`. Antwortbeispiel: `{"status": "ok"}` und

`{"status": "ready", "ready": true}`. Fehlerbeispiel: curl-Exits ungleich Null für Transportfehler oder Nicht-2xx-Antworten, wenn `-f` verwendet wird.

```
curl -fsS "$BPM_BASE_URL/health"
curl -fsS "$BPM_BASE_URL/health/ready"
```

3. Wiederverwendung von Profillebenszyklus-Snippets zum Erstellen, Auflisten, Lesen und für geschützte Aktualisierungen.

Anforderungsbeispiel: POST `$BPM_BASE_URL/api/profiles`, GET `$BPM_BASE_URL/api/profiles`, GET `$BPM_BASE_URL/api/profiles/$PROFILE_ID` und PATCH `$BPM_BASE_URL/api/profiles/$PROFILE_ID`. Antwortbeispiel: 201 ProfileRead, 200 ProfileRead[], 200 ProfileRead und ein inkrementiertes revision. Fehlerbeispiele: 400, 404, 409 veraltetes expected_revision oder 422.

```
PROFILE_JSON="$(curl -fsS -H 'Content-Type: application/json' \
-d '{"name":"docs-api-example","schema_version":"release-153","flags":{"DisableTelemetry":true}}' \
"$BPM_BASE_URL/api/profiles")"
PROFILE_ID="$(python -c 'import json,sys; print(json.load(sys.stdin)["id"])' <<<"$PROFILE_JSON")"
PROFILE_REVISION="$(python -c 'import json,sys; print(json.load(sys.stdin)["revision"])'
<<<"$PROFILE_JSON")"

curl -fsS "$BPM_BASE_URL/api/profiles?q=docs-api-example&lifecycle=active&limit=50&offset=0"
curl -fsS "$BPM_BASE_URL/api/profiles/$PROFILE_ID"
curl -fsS -X PATCH -H 'Content-Type: application/json' \
-d '{"flags":
{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":$PROFILE_REVISION}' \
"$BPM_BASE_URL/api/profiles/$PROFILE_ID"
```

4. Wiederverwendung von Validierungs- und JSON-Import-Snippets mit expliziter Nutzlastform.

Anforderungsbeispiel: POST `$BPM_BASE_URL/api/validate/release-153` und POST `$BPM_BASE_URL/api/profiles/import/firefox/policies.json`. Antwortbeispiel: 200 `{"ok":true,"profile":"release-153"}` und 201 ProfileRead. Fehlerbeispiele: 400, 409, 415, 422 oder 503.

```
curl -fsS -H 'Content-Type: application/json' \
-d '{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}' \
"$BPM_BASE_URL/api/validate/$BPM_SCHEMA_CHANNEL"

curl -fsS -H 'Content-Type: application/json' \
-d '{"name":"docs-json-import-example","schema_version":"release-153","compliance":{"source":"docs-devops-template"},"document":{"policies":{"DisableTelemetry":true}}}' \
"$BPM_BASE_URL/api/profiles/import/firefox/policies.json"
```

5. Wiederverwendung mehrteiliger Import- und Export-Snippets für die dateibasierte Übergabe.

Anforderungsbeispiel: multipart/form-data-Upload auf POST `$BPM_BASE_URL/api/profiles/import/firefox/policies.json`, gefolgt von GET `$BPM_BASE_URL/api/export/profiles/$PROFILE_ID/firefox/policies.json?pretty=1`. Antwortbeispiel: 201 ProfileRead für den Import und 200 application/json mit dem kanonischen Firefox policies für den Export. Fehlerbeispiel: 404 fehlendes Profil oder 422 ungültiges Profil ID.

```
curl -fsS \
-F "name=docs-multipart-import-example" \
-F "schema_version=$BPM_SCHEMA_CHANNEL" \
-F "compliance={\"source\":\"docs-devops-template\"}" \
-F "file=@$BPM_JSON_IMPORT_PATH;type=application/json" \
"$BPM_BASE_URL/api/profiles/import/firefox/policies.json"
```

```
curl -fsS "$BPM_BASE_URL/api/export/profiles/$PROFILE_ID/firefox/policies.json?pretty=1"
```

6. Verwenden Sie einen kompakten Python-Helfer, wenn die Integration wiederverwendbare Antwortzusicherungen benötigt.

Anforderungsbeispiel: `session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)`. Antwortbeispiel: Der Helfer löst sofort aus, es sei denn, der Status und der JSON-Text stimmen mit der Behauptung überein. Fehlerbeispiel: Assertionsfehler, Timeout-Ausnahmen, veraltete `expected_revision` oder strukturierte API-Fehler müssen vom integrierenden Produkt aufgezeichnet werden.

```
import os
import requests

base_url = os.environ["BPM_BASE_URL"].rstrip("/")
schema_channel = os.environ.get("BPM_SCHEMA_CHANNEL", "release-153")
session = requests.Session()

ready = session.get(f"{base_url}/health/ready", timeout=10)
ready.raise_for_status()
assert ready.json() == {"status": "ready", "ready": True}

payload = {"document": {"policies": {"DisableTelemetry": True, "BlockAboutConfig": True}}}
validation = session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)
validation.raise_for_status()
assert validation.json() == {"ok": True, "profile": schema_channel}

created = session.post(
    f"{base_url}/api/profiles",
    json={"name": "docs-python-example", "schema_version": schema_channel, "flags":
payload["document"]["policies"]},
    timeout=10,
)
created.raise_for_status()
profile = created.json()
assert profile["schema_version"] == schema_channel

exported = session.get(
    f"{base_url}/api/export/profiles/{profile['id']}/firefox/policies.json?pretty=1",
    timeout=10,
)
exported.raise_for_status()
assert exported.json() == {"policies": payload["document"]["policies"]}
```

WARNUNG:

Behalten Sie den Antwortstatus, das Profil ID, die Revision, den Schemakanal, den exportierten Dokument-Hash und das Behauptungsergebnis in externen Beweisen bei. Kopieren Sie keine echten Hostnamen, Geheimnisse, API-Schlüssel, Geheimwerte oder Produktionskennungen in Dokumentationsbeispiele.

Das integrierende Produkt verfügt über wiederverwendbare curl- und Python-Beispiele, die sicher kopiert werden können, unabhängig von Bereitstellungshosts, mit dem aktuellen BPM-API-Verhalten synchronisiert und an explizite Antwortzusicherungen gebunden sind.

Kombinieren Sie diese Beispiele mit den Betriebsanleitungen Pull/List/Read, Validate-Before-Apply, Import-Prüfung-Export und Failure-Audit, wenn Sie vollständige Arbeitsabläufe für externe Steuerungsprodukte erstellen.

Profillebenszyklusdaten synchronisieren

Verwenden Sie Profillisten und Statistiken, lesen, erstellen und aktualisieren Sie Endpunkte, ohne Massen- oder Transaktionsverhalten anzunehmen.

Legen Sie `$BPM_BASE_URL` fest und lesen Sie `/openapi.json` für die aktuellen Formen `ProfileCreate`, `ProfileUpdate` und `ProfileRead`.

Diese Aufgabe umfasst API-PROFILE-001, API-PROFILE-002, API-PROFILE-003, API-PROFILE-004 und API-PROFILE-005. Store hat `id` und `revision` zurückgegeben; Schließen Sie Ihre Identität nicht allein aus dem Namen ab.

1. Listen Sie Profile mit Filtern auf, bevor Sie ein Profil zum Synchronisieren auswählen.

Anforderungsbeispiel: `GET $BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`. Antwortbeispiel: `200 ProfileRead[]`. Fehlerbeispiel: 422 für ungültige typisierte oder bereichsbezogene Abfrageeingabe.

2. Lesen Sie Profilstatistiken für denselben Filterkontext, wenn Sie Zählungen benötigen.

Anforderungsbeispiel: `GET $BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`. Antwortbeispiel: `200 {"filtered":1,"total":3}`. Fehlerbeispiel: 422 für ungültige Abfrageeingabe.

3. Lesen Sie das ausgewählte Profil anhand der Ganzzahl ID und schließen Sie archivierte Datensätze nur bei Bedarf ein.

Anforderungsbeispiel: `GET $BPM_BASE_URL/api/profiles/42?include_deleted=false`. Antwortbeispiel: `200 ProfileRead` mit `id`, `revision`, Zeitstempel, Lebenszyklus und Validierungsstatus. Fehlerbeispiel: 404 für ein fehlendes oder nicht sichtbares Profil; 422 für ungültigen Pfad oder ungültige Abfrageeingabe.

4. Erstellen Sie nur dann ein normalisiertes BPM-Profil, wenn die Integration den neuen Profilnamen besitzt.

Anforderungsbeispiel: `POST $BPM_BASE_URL/api/profiles` mit JSON `{"name":"corp-baseline","schema_version":"release-153","flags":{},"compliance":{}}`. Antwortbeispiel: `201 ProfileRead`. Fehlerbeispiele: 400 ungültiges Schema/Profil, 409 doppelter Name oder 422 Anforderung oder Richtlinienvalidierung.

5. Aktualisieren Sie veränderliche Felder mit PATCH und senden Sie `expected_revision`, wenn Sie optimistische Parallelität benötigen.

Anforderungsbeispiel: `PATCH $BPM_BASE_URL/api/profiles/42` mit JSON `{"description":"Reviewed by control plane","expected_revision":7}`. Antwortbeispiel: `200 ProfileRead` mit erhöhter Revision nach einer Materialaktualisierung. Fehlerbeispiele: 400 ungültiges Schema/Profil, 404 fehlendes oder archiviertes Profil, 409 veraltetes `expected_revision` oder 422 Validierungsfehler.

6. Zeichnen Sie das synchronisierte Profil ID, die Revision, den Validierungsstatus, den Schemakanal und alle undurchsichtigen Compliance-Metadaten im integrierenden Produkt auf.

WARNUNG:

BPM bietet keine Multioperationstransaktion. Wenn ein späterer Schritt fehlschlägt, ist das integrierende Produkt Eigentümer der Wiederherstellungs-, Prüfdatensätze und Wiederholungsentscheidungen.

Die Integration verfügt über einen reproduzierbaren Profilsynchronisierungspfad für Listen-, Statistik-, Lese-, Erstellungs- und Aktualisierungsvorgänge.

Verwenden Sie die destruktive Lebenszyklusaufgabe vor Archivierungs-, Wiederherstellungs-, dauerhaften Lösch- oder Rücksetzvorgängen.

Profile archivieren, wiederherstellen, dauerhaft löschen oder zurücksetzen

Verwenden Sie destruktive Profillebenszyklus-Endpunkte mit expliziten Überprüfungs- und Wiederherstellungsdatensätzen.

Bestätigen Sie das Zielprofil ID, den aktuellen Lebenszyklusstatus und den externen Genehmigungsdatensatz, bevor Sie destruktive Endpunkte aufrufen.

Diese Aufgabe umfasst API -PROFILE-006, API -PROFILE-007, API -PROFILE-008 und API -PROFILE-009. BPM verfügt über kein Massenbestätigungstoken, keinen Rückgängig-Endpunkt für dauerhaftes Löschen und kein vorgangsübergreifendes Transaktions-Rollback.

1. Archivieren Sie ein Profil, wenn Sie es aus aktiven Arbeitsabläufe entfernen müssen, während Sie es zur Überprüfung aufbewahren.

WARNUNG:

Archive ist eine destruktive Lebenszyklusänderung für aktive Arbeitsabläufe. Notieren Sie vor dem Anruf den Genehmigungsgrund und das Profil ID.

Anforderungsbeispiel: DELETE \$BPM_BASE_URL/api/profiles/42. Antwortbeispiel: 204 ohne JSON-Text. Fehlerbeispiele: 404 fehlendes Profil oder 422 ungültige Pfadeingabe.

2. Stellen Sie ein archiviertes Profil erst wieder her, nachdem überprüft wurde, ob der Name und der Profilzweck noch gültig sind.

Anforderungsbeispiel: POST \$BPM_BASE_URL/api/profiles/42/restore. Antwortbeispiel: 200 ProfileRead. Fehlerbeispiele: 404-Profil kann nicht wiederhergestellt oder gefunden werden; 422 ungültige Pfadeingabe.

3. Löschen Sie ein Profil nur dann dauerhaft, wenn die externe Aufbewahrungsrichtlinie das Entfernen zulässt.

WARNUNG:

Permanentes Löschen entfernt ein aktives oder archiviertes Profil und hat keinen BPM-Rückgängig-Endpunkt. Exportieren oder erfassen Sie erforderliche Nachweise vor dem Anruf.

Anforderungsbeispiel: DELETE \$BPM_BASE_URL/api/profiles/42/hard. Antwortbeispiel: 204 ohne JSON-Text. Fehlerbeispiele: 404 fehlendes Profil oder 422 ungültige Pfadeingabe.

4. Setzen Sie die Bibliothek nur in Einweg- oder ausdrücklich genehmigten Umgebungen zurück.

WARNUNG:

Beim Zurücksetzen wird jedes Profil in der Bibliothek dauerhaft gelöscht und es ist derzeit kein Bestätigungstoken erforderlich. Verwenden Sie es niemals als routinemäßige Synchronisierungsverknüpfung.

Anforderungsbeispiel: DELETE \$BPM_BASE_URL/api/profiles/reset. Antwortbeispiel: 200 {"deleted":3}. Fehlerverhalten: Derzeit ist kein Anwendungsfehler gemeldet; Datenbankfehler nutzen das Framework-Handling.

5. Listen Sie die Profile nach jedem destruktiven Aufruf erneut auf und lesen Sie ggf. den betroffenen Datensatz.

Anforderungsbeispiel: GET \$BPM_BASE_URL/api/profiles?lifecycle=all. Antwortbeispiel: 200 ProfileRead[]. Fehlerbeispiel: 422 ungültige Abfrageeingabe. Verwenden Sie diese Überprüfung, um den Bestandsstatus des integrierenden Produkts zu aktualisieren.

6. Schreiben Sie den externen Prüfdatensatz mit Vorgang ID, Methode HTTP, Pfad, Profil ID, Genehmiger, Grund, Zeitstempel, Antwortstatus und Wiederherstellungsentscheidung.

Erwartetes Ergebnis: Ein späterer Prüfer kann Archivierungs-, Wiederherstellungs-, Hard-Delete- und Reset-Aktionen unterscheiden und erkennen, warum die Integration den Vorgang als sicher betrachtete.

Zerstörerische Profillebenszyklusvorgänge werden mit Anforderungs-, Antwort-, Fehler-, Warn- und Wiederherstellungsnachweisen dokumentiert.

Wenn der Genehmigungsdatensatz fehlt, stoppen Sie den Integrationsworkflow, bevor Sie das dauerhafte Löschen oder Zurücksetzen aufrufen.

Firefox policies.json über die API importieren

Erstellen Sie ein normalisiertes BPM-Profil aus einem kanonischen Firefox Enterprise policies.json-Dokument mit JSON oder mehrteiliger Eingabe.

Legen Sie \$BPM_BASE_URL fest, wählen Sie einen unterstützten Schemakanal wie release-153 und bereiten Sie ein vollständiges Firefox-Dokument mit einem policies-Objekt der obersten Ebene vor.

Diese Aufgabe umfasst API-FF-001. Der Import akzeptiert application/json mit FirefoxPoliciesJsonImportRequest oder multipart/form-data mit einem file-Feld. BPM validiert das kanonische Firefox-Dokument, normalisiert policies in das Profil flags und speichert optionale undurchsichtige compliance-Metadaten im BPM-Profil.

1. Senden Sie eine JSON-Importanfrage, wenn das integrierende Produkt das Dokument bereits im Speicher hat.

Anforderungsbeispiel: POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json mit Content-Type: application/json und JSON {"name": "api-json-import", "description": "Imported by control product", "schema_version": "release-153", "compliance": {"framework": "cis", "layer": "cis_l1"}, "document": {"policies": {"DisableTelemetry": true, "BlockAboutConfig": true}}}.

2. Verwenden Sie den mehrteiligen Import, wenn das Quellsystem eine Datei übergibt.

Anforderungsbeispiel: POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json mit multipart/form-data, den Feldern name=api-multipart-import, schema_version=release-153, optional description, optional compliance={"framework": "cis", "layer": "cis_l2"} als JSON-Zeichenfolge und file=@policies.json;type=application/json. Runtime akzeptiert auch ein mehrteiliges document-Feld, aber das mehrteilige OpenAPI-Schema deklariert derzeit file; Verwenden Sie file für neue Beispiele.

3. Lesen Sie das erstellte BPM-Profil aus der 201 ProfileRead-Antwort.

Antwortbeispiel: 201 ProfileRead mit schema_version="release-153", flags.DisableTelemetry=true, flags.BlockAboutConfig=true, gespeichertem compliance, id und revision. Speichern Sie das zurückgegebene Profil ID, anstatt die Identität aus dem Profilnamen abzuleiten.

4. Behandeln Sie Importfehler als Fehler bei der Profilerstellung und gehen Sie nicht von einem Teilprofilstatus aus.

Fehlerbeispiele: 400 für fehlerhaftes JSON, falsche Dokumentform oder nicht unterstützten Schemakanal; 409 für doppelten Profilnamen; 415 für nicht unterstützten Medientyp; 422 für Anfrage- oder Firefox-Richtlinienvvalidierungsfehler.

5. Halten Sie kanonische Firefox- und normalisierte BPM-Formen im Integrationscode getrennt.

Die Anforderung document.policies ist die Firefox-Bereitstellungsgrenze. Die flags des erstellten Profils sind das normalisierte Modell von BPM. Senden Sie kein flags-Objekt der obersten Ebene an diesen Importendpunkt.

6. Importnachweise im integrierenden Produkt aufzeichnen.

WARNUNG:

Import erstellt ein BPM-Profil, stellt jedoch keine Firefox-Einstellungen bereit, prüft die Laufzeiterzwingung nicht und stellt keinen Idempotenzschlüssel bereit. Zeichnen Sie den Quelldokument-Hash, den Schemakanal, das zurückgegebene Profil ID, die Revision, die Compliance-Metadaten und die Entscheidung zur Wiederherstellung doppelter Namen extern auf.

Das Firefox-Dokument `policies.json` wird als normalisiertes BPM-Profil dargestellt, das von ID überprüft, validiert, exportiert und verfolgt werden kann.

Verwenden Sie die Exportaufgabe, um das gespeicherte Profil wieder im kanonischen Firefox-JSON zu rendern, bevor Sie es an die Bereitstellungstools übergeben.

Die kanonische Firefox policies.json über die API exportieren

Rendern Sie ein BPM-Profil als Firefox Enterprise-Bereitstellungsdokument mit optionalen Download- und Pretty-Print-Parametern.

Speichern Sie das BPM-Profil ID, das durch Erstellen oder Importieren zurückgegeben wird. Wenn das Profil archiviert ist, entscheiden Sie explizit, ob die Integration es mit `include_deleted=true` exportieren darf.

Diese Aufgabe umfasst API-FF-002. Export liest BPMs normalisiertes flags und gibt kanonisches Firefox-JSON mit einem policies-Objekt der obersten Ebene zurück. BPM-Profilmetadaten und undurchsichtige compliance-Metadaten werden nicht in das Firefox-Bereitstellungsdokument exportiert.

1. Exportieren Sie das aktive Profil als kanonisches Firefox-JSON.

Anforderungsbeispiel: GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json`.

Antwortbeispiel: 200 application/json mit JSON `{"policies":`

`{"DisableTelemetry":true,"BlockAboutConfig":true}}}`.

2. Download-Modus hinzufügen, wenn der Anrufer einen Anhang-Dateinamen benötigt.

Anforderungsbeispiel: GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?`

`download=1`. Antwortbeispiel: 200 application/json plus Content-Disposition: attachment;

`filename="profile-42-policies.json"`. Der JSON-Body ist dasselbe Bereitstellungsdokument.

3. Wählen Sie Formatierungsparameter bewusst aus.

Anforderungsbeispiel: GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?`
`pretty=1` wählt die Einrückung von zwei Leerzeichen aus, wenn indent weggelassen wird.

Anforderungsbeispiel: GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?`
`indent=0` verwendet den expliziten Einrückungsparameter. Fehlerbeispiel: 422 für ungültige eingeschränkte Abfragewerte.

4. Exportieren Sie ein archiviertes Profil nur, wenn der externe Arbeitsablauf einen Überprüfungsgrund hat.

Anforderungsbeispiel: GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?`
`include_deleted=true&download=1&pretty=1`. Antwortbeispiel: 200 application/json.

Fehlerbeispiele: 404, wenn ein fehlendes oder archiviertes Profil nicht sichtbar ist; 422 für ungültigen Pfad oder ungültige Abfrageeingabe.

5. Validieren Sie das exportierte Dokument, bevor Sie es an ein anderes System übergeben.

Anforderungsbeispiel: POST `$BPM_BASE_URL/api/validate/release-153` mit JSON `{"document":`
`{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Antwortbeispiel: 200
`{"ok":true,"profile":"release-153"}`. Die Validierung bestätigt die Schemaform, nicht den Erfolg der Firefox-Laufzeitbereitstellung.

6. Zeichnen Sie das genaue exportierte Artefakt extern auf.

WARNUNG:

Der Export hat kein ETag, keinen Vorbedingungsheader, keinen Massenendpunkt und keinen Transaktionswrapper. Zeichnen Sie das Profil ID, die vor dem Export beobachtete Revision, den Schemakanal, die Abfrageparameter, den Hash des exportierten Dokuments und die Downstream-Übergabeentscheidung außerhalb von BPM auf.

Die Integration verfügt über ein kanonisches Firefox-Artefakt `policies.json`, das reine BPM-Metadaten ausschließt und an Bereitstellungstools bereitgestellt werden kann.

Wenn die Validierung fehlschlägt oder sich die Profilrevision zwischen Lesen und Export geändert hat, stoppen Sie die Übergabe und aktualisieren Sie den Profilstatus.

Die Firefox-Kandidatendatei policies.json über die API validieren

Verwenden Sie den Validierungsendpunkt als Preflight-Gate für Kandidatendokumente für Firefox Enterprise policies.json und strukturierte Fehlerbehandlung.

Wählen Sie den Zielschemakanal, z. B. release-153 oder esr-140.13, und bereiten Sie einen ValidationRequest-Body mit einem vollständigen Firefox-Dokument in document.policies vor.

Diese Aufgabe umfasst API-VAL-001. BPM validiert Kandidateninhalte von policies.json vor dem Import, der Exportübergabe oder einem externen Richtlinienüberprüfungs-Gate. Aus Kompatibilitätsgründen wird weiterhin eine einfache Richtlinienzuordnung akzeptiert, neue Integrationsbeispiele verwenden jedoch die vollständige Firefox-policies.json-Form.

1. Senden Sie eine erfolgreiche Validierungsanforderung für den ausgewählten Schemakanal.

Anforderungsbeispiel: POST \$BPM_BASE_URL/api/validate/release-153 mit JSON {"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}. Antwortbeispiel: 200 {"ok":true,"profile":"release-153"}.

2. Behandeln Sie ok=false als fehlgeschlagenes Gate, auch wenn der Status von HTTP 200 ist.

Anforderungsbeispiel: POST \$BPM_BASE_URL/api/validate/release-153 mit JSON {"document":123}. Antwortbeispiel: 200 {"ok":false,"profile":"release-153","detail":"Expected object with policy mappings","error":"Expected object with policy mappings"}. Überprüfen Sie den Antworttext und nicht nur den HTTP-Status.

3. Behandeln Sie fehlerhafte kanonische Firefox-Dokumentfehler als Anforderungsformfehler.

Anforderungsbeispiel: POST \$BPM_BASE_URL/api/validate/release-153 mit JSON {"document":{"policies":[]}}. Fehlerbeispiel: 400 mit detail.message="Firefox policies.json validation failed" und detail.issues[].path zeigen auf policies.

4. Nicht unterstützte Schemakanäle ablehnen, bevor den Validierungsergebnissen vertraut wird.

Anforderungsbeispiel: POST \$BPM_BASE_URL/api/validate/beta-999 mit JSON {"document":{"policies":{"DisableTelemetry":true}}}. Fehlerbeispiel: 404 mit Detailtext Unknown profile 'beta-999'. Ein registriertes, aber nicht verfügbares Schema gibt 503 zurück.

5. Ordnen Sie Fehler bei der Richtlinienvvalidierung dem umsetzbaren Feedback der Prüfer zu.

Anforderungsbeispiel: POST \$BPM_BASE_URL/api/validate/release-153 mit JSON {"document":{"policies":{"Proxy":{"Mode":"bogus"}}}}. Fehlerbeispiel: 422 mit detail.message="Policy validation failed" und den Problemfeldern policy, path und message.

6. Zeichnen Sie die Validierungsentscheidung im integrierenden Produkt auf.

WARNUNG:

Die Validierung ist nur ein Schema- und Policy-Shape-Gate. Es stellt weder Firefox bereit, noch prüft es die Durchsetzung der Laufzeit, authentifiziert keine Aufrufer, reserviert keine Profilrevision und stellt keinen Idempotenzschlüssel bereit. Speichern Sie den Kanal, den Hash des Kandidatendokuments, den Antwortstatus, den ok-Wert, strukturierte Probleme und die Entscheidung des Prüfers extern.

Die Integration kann unsichere Kandidaten-policies.json-Übergaben blockieren und strukturierte Beweise für Erfolgs- und Misserfolgsentscheidungen bewahren.

Wenn die Validierung erfolgreich ist, fahren Sie mit dem Import, der Profilaktualisierung oder der Exportübergabe unter Verwendung desselben Schemakanals und aufgezeichneten Kandidaten-Hash fort.

Lebenszyklusabläufe ausführen und Fehler beheben

Ein Pull-, Vergleichs- und Aktualisierungsszenario für ein Steuerungsprodukt ausführen

Verwenden Sie das vorhandene Profil und die Validierung API-Schnittstellen, um den BPM-Status abzurufen, ihn extern zu vergleichen und ein Profil mit expliziten Eigentums- und Parallelitätsnachweisen zu aktualisieren.

Legen Sie `$BPM_BASE_URL` fest und entscheiden Sie, welches externe Kontrollprodukt Vergleichslogik, Sollzustandsberechnung, Wiederholungsrichtlinie und Prüfdatensätze besitzt. BPM stellt weder eine Vergleichs-API noch einen Transaktions-Wrapper bereit.

Dieses Szenario verwendet API-PROFILE-001, API-PROFILE-003, API-PROFILE-005 und API-VAL-001. BPM besitzt den gespeicherten Profilstatus und die Validierung. Das integrierende Produkt besitzt den gewünschten Status, die Vergleichsausgabe, die Genehmigung des Prüfers und Entscheidungen zu Wiederholungsversuchen.

1. Rufen Sie Kandidatenprofile aus der Bibliothek mit denselben Filterregeln ab, die das Kontrollprodukt für den Besitz verwendet.

Anforderungsbeispiel: `GET $BPM_BASE_URL/api/profiles?`

`q=corp&lifecycle=active&limit=50&offset=0`. Antwortbeispiel: `200 ProfileRead[]`.

Fehlerbeispiel: 422 für ungültige Abfrageeingabe.

2. Jedes ausgewählte Profil nach ID lesen und seinen aktuellen `revision` speichern.

Anforderungsbeispiel: `GET $BPM_BASE_URL/api/profiles/42`. Antwortbeispiel: `200 ProfileRead` mit `flags`, `compliance`, `schema_version` und `revision`. Fehlerbeispiele: 404 fehlendes oder archiviertes Profil; 422 ungültige Pfad eingabe.

3. Vergleichen Sie das gezogene Profil mit dem externen gewünschten Zustand außerhalb von BPM.

Erwartetes Ergebnis: Das integrierende Produkt erfasst geänderte Richtlinienschlüssel, unveränderte Schlüssel, fehlende Schlüssel, Prüfer und Quelle des gewünschten Status. Gehen Sie nicht davon aus, dass ein BPM-Vergleichsendpunkt, ein Marktplatz-Connector oder ein zertifizierter Arbeitsablauf vorhanden ist.

4. Validieren Sie das Firefox-Kandidatendokument, bevor Sie das Update anwenden.

Anforderungsbeispiel: `POST $BPM_BASE_URL/api/validate/release-153` mit `JSON {"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Antwortbeispiel: `200 {"ok":true,"profile":"release-153"}`. Fehlerbeispiele: 400, 404, 422 oder 503, je nach Dokumentform, Kanal, Richtliniengültigkeit und Schemaverfügbarkeit.

5. Patchen Sie das BPM-Profil nur mit der Revision, die vor dem Vergleich beobachtet wurde.

Anforderungsbeispiel: `PATCH $BPM_BASE_URL/api/profiles/42` mit `JSON {"flags":{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":`

`{"source":"control-product","decision":"approved"}}`. Antwortbeispiel: 200 ProfileRead mit inkrementiertem revision. Fehlerbeispiele: 400, 404, 409 veraltetes expected_revision oder 422.

6. Behandeln Sie Fehler ohne automatische Wiedergabe.

WARNUNG:

Wenn die Validierung fehlschlägt, die Revision veraltet ist oder das Aktualisierungsergebnis nicht mit dem beabsichtigten gewünschten Status übereinstimmt, stoppen Sie den Arbeitsablauf, lesen Sie den BPM-Status erneut und erstellen Sie einen externen Prüfdatensatz. BPM bietet keine Multioperationstransaktion, kein automatisches Rollback, keinen Idempotenzschlüssel und keinen sicheren Wiederholungsvertrag.

Das Kontrollprodukt verfügt über einen ausführbaren Pull-/Vergleichs-/Aktualisierungsfluss mit explizitem Zustandseigentum, Validierungsnachweisen, Revisionsschutz und Fehlerwiederherstellungsgrenzen.

Wenn das aktualisierte Profil an anderer Stelle bereitgestellt werden muss, fahren Sie mit der Firefox-Exportaufgabe fort und behalten Sie das Profil ID, die Revision, die Validierungsantwort und den Genehmigungsdatensatz bei.

Ein Import-, Prüfungs-, Export- und Compliance-Übergabeszenario ausführen

Verwenden Sie die vorhandenen Integritäts-, Import-, Validierungs-, Profilierungs- und Exportfunktionen API-Schnittstellen, um ein überprüftes Firefox-Dokument mit Konformitätsnachweisen zu übergeben.

Legen Sie `$BPM_BASE_URL` fest, wählen Sie den Schemakanal aus und entscheiden Sie, welches System Eigentümer des Quelldokuments, des Compliance-Scan-Ergebnisses, der Prüfergenehmigung, des exportierten Artefakts und der Wiederherstellungsentscheidung ist.

Dieses Szenario verwendet API-HEALTH-001, API-HEALTH-002, API-FF-001, API-PROFILE-003, API-VAL-001 und API-FF-002. BPM speichert das normalisierte Profil und undurchsichtige Compliance-Metadaten. Das integrierende Produkt verfügt über Scan-Interpretation und Downstream-Bereitstellung.

1. Beginnen Sie vor der Übergabesequenz mit Gesundheit und Bereitschaft.

Anforderungsbeispiele: GET `$BPM_BASE_URL/health` und GET `$BPM_BASE_URL/health/ready`.

Antwortbeispiele: 200 `{"status": "ok"}` und 200 `{"status": "ready", "ready": true}`.

Fehlerbeispiel: Transport- oder Gatewayfehler werden außerhalb von BPM behandelt, da derzeit kein Anwendungsfehler deklariert ist.

2. Importieren Sie das überprüfte Firefox-Dokument mit an das BPM-Profil angehängten Compliance-Metadaten.

Anforderungsbeispiel: POST `$BPM_BASE_URL/api/profiles/import/firefox/policies.json` mit JSON `{"name": "reviewed-corp-baseline", "schema_version": "release-153", "compliance": {"scanner": "external-control", "result": "passed", "ticket": "SEC-42"}, "document": {"policies": {"DisableTelemetry": true, "BlockAboutConfig": true}}}`. Antwortbeispiel: 201 ProfileRead. Fehlerbeispiele: 400, 409, 415 oder 422.

3. Lesen Sie das erstellte Profil und zeichnen Sie die BPM-Besitzfelder getrennt von externen Beweisen auf.

Anforderungsbeispiel: GET `$BPM_BASE_URL/api/profiles/42`. Antwortbeispiel: 200 ProfileRead mit id, revision, flags, schema_version und undurchsichtiges compliance. Fehlerbeispiele: 404 oder 422.

4. Validieren Sie das kanonische Firefox-Dokument, das übergeben wird.

Anforderungsbeispiel: POST `$BPM_BASE_URL/api/validate/release-153` mit JSON `{"document": {"policies": {"DisableTelemetry": true, "BlockAboutConfig": true}}}`. Antwortbeispiel: 200 `{"ok": true, "profile": "release-153"}`. Fehlerbeispiele: 400, 404, 422 oder 503.

5. Exportieren Sie das BPM-Profil als kanonisches Firefox-Bereitstellungsdokument.

Anforderungsbeispiel: GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1`. Antwortbeispiel: 200 application/json mit Content-Disposition und policies der obersten Ebene. BPM-Profilmetadaten und Compliance-Metadaten werden nicht exportiert.

6. Stoppen Sie bei Nichtübereinstimmungen und schreiben Sie den Fehlerwiederherstellungsdatensatz.

WARNUNG:

Wenn Import-, Validierungs-, Rücklese- oder Exportnachweise vom genehmigten Quelldokument abweichen, versuchen Sie es nicht blind erneut. Speichern Sie Quell-Hash, Profil ID, Revision, Validierungstext, Export-Hash, Scanner-Ergebnis, Prüfer, Antwortstatus und die Wiederherstellungsentscheidung außerhalb von BPM.

Das integrierende Produkt verfügt über einen ausführbaren Import-Überprüfung-Export-Ablauf mit Compliance-Übergabe, kanonischem Export und expliziten Wiederherstellungsnachweisen.

Verwenden Sie Bereitstellungstools außerhalb von BPM, um die exportierte `policies.json` anzuwenden; Die BPM-Validierung beweist nicht die Durchsetzung der Firefox-Laufzeit.

BPM-Profil aus einem externen Steuerungsprodukt abrufen und prüfen

Erstellen Sie die Pull-/Listen-/Leseseite einer Kontrollproduktintegration mit expliziten Besitzverhältnissen, Filtern und Beweisgrenzen.

Legen Sie `$BPM_BASE_URL` fest. Entscheiden Sie, welches externe Kontrollprodukt die Bestandsauswahl, den Vergleichsspeicher, die Wiederholungsrichtlinie und die Prüfnachweise besitzt. BPM besitzt nur die von seiner API zurückgegebenen Profildatensätze.

Dieser Arbeitsablauf verwendet API-PROFILE-001, API-PROFILE-002 und API-PROFILE-003. Es ist der sichere schreibgeschützte Einstiegspunkt vor den Arbeitsabläufen Vergleichen/Aktualisieren, Validieren-vor-Anwenden, Importieren-Überprüfen-Exportieren oder Compliance-Metadatenübergabe.

1. Bestätigen Sie den Kontrollproduktumfang, bevor Sie den BPM-Status lesen.

Erfassen Sie den externen Eigentümer, den Zweck, den Schemakanal, die Filterregeln und ob archivierte Profile ausgeschlossen sind. Verwenden Sie `lifecycle=active`, es sei denn, ein Wiederherstellungsvorgang benötigt explizit `include_deleted=true`.

2. Listen Sie Profile mit deterministischen Filtern und Paging auf.

Anforderungsbeispiel: `GET $BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`. Antwortbeispiel: `200 ProfileRead[]`. Fehlerbeispiel: 422 für ungültige Abfrageparameter.

3. Erfassen Sie optionale Bestandszählungen getrennt von den Profildaten.

Anforderungsbeispiel: `GET $BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`. Antwortbeispiel: `200 application/json`. Speichern Sie Zählnachweise als beratenden Bestandskontext, nicht als Sperre oder Transaktionsgrenze.

4. Lesen Sie jedes ausgewählte Profil vor dem Vergleich um ID.

Anforderungsbeispiel: `GET $BPM_BASE_URL/api/profiles/42?include_deleted=false`. Antwortbeispiel: `200 ProfileRead` mit `id`, `revision`, `schema_version`, `flags` und `compliance`. Fehlerbeispiele: 404 oder 422.

5. Behalten Sie einen Lese-Snapshot außerhalb von BPM für das Kontrollprodukt bei.

Store-Profil ID, Revision, Schemakanal, Filterabfrage, Lesezeitstempel, Quell-URL, ausgewählte Richtlinienschlüssel und der Kontrollproduktlauf ID. Speichern Sie keine Geheimnisse oder privaten Hostnamen in gemeinsam genutzten Dokumentationsbeispielen.

6. Stoppen, wenn das Pull-Ergebnis nicht vertrauenswürdig ist.

WARNUNG:

Wenn Paging, Filterung oder Rücklesebeweise unvollständig sind, stoppen Sie vor der Aktualisierung. BPM stellt keine Cursor-Paginierung, Snapshot-Isolierung, Vergleichssperren, Webhooks oder einen sicheren Wiedergabevertrag für Leseworkflows zur Verfügung.

Das externe Kontrollprodukt verfügt über einen reproduzierbaren Pull-/Listen-/Lese-Snapshot und weiß, welche Beweise zu BPM und welche Beweise zum externen System gehören.

Fahren Sie mit „Validieren vor Anwenden“ oder „Vergleichen/Aktualisieren“ erst fort, nachdem das externe System das ausgewählte Profil Kennungen und die Revisionen aufgezeichnet hat.

Eine Profilaktualisierung mit `expected_revision` validieren und anwenden

Führen Sie einen Validierungs-First-Update-Arbeitsablauf aus, der den BPM-Status mit der aktuellen Revision schützt und bei veralteten oder ungültigen Daten stoppt.

Das externe Produkt hat das Zielprofil bereits gelesen und seine aktuelle `revision` gespeichert. Bereiten Sie das Firefox-Kandidatendokument `policies.json` vor, bevor Sie einen Profilpatch senden.

Dieser Arbeitsablauf verwendet API-VAL-001, API-PROFILE-003 und API-PROFILE-005. Es dokumentiert den aktuellen optimistischen Update-Pfad; BPM stellt weder ETags noch Vorbedingungsheader, Idempotenzschlüssel oder Transaktionen mit mehreren Operationen bereit.

1. Erstellen Sie das Firefox-Kandidatendokument außerhalb von BPM.

Erfassen Sie die gewünschten Richtlinien Schlüssel, das Quellsystem, den Prüfer, das Ticket und den Schemakanal. Beispielkandidat: `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`.

2. Validieren Sie den Kandidaten, bevor Sie das Profil patchen.

Anforderungsbeispiel: `POST $BPM_BASE_URL/api/validate/release-153`. Antwortbeispiel: `200 {"ok":true,"profile":"release-153"}`. Fehlerbeispiele: 400, 404, 422 oder 503.

3. Lesen Sie das Profil noch einmal, unmittelbar bevor Sie das Update anwenden, wenn das Genehmigungsfenster lang ist.

Anforderungsbeispiel: `GET $BPM_BASE_URL/api/profiles/42`. Antwortbeispiel: `200 ProfileRead`. Wenn sich `revision` geändert hat, führen Sie den Vergleich und die Genehmigung erneut durch, anstatt den veralteten Zustand zu patchen.

4. Patchen Sie nur veränderbare Felder mit der beobachteten Revision.

Anforderungsbeispiel: `PATCH $BPM_BASE_URL/api/profiles/42` mit JSON `{"flags":{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":{"source":"control-product","decision":"approved"}}`. Antwortbeispiel: `200 ProfileRead` mit inkrementiertem `revision`.

5. Behandeln Sie Konfliktreaktionen als Stoppsignale.

Fehlerbeispiele: 400 ungültige Nutzlast, 404 fehlendes Profil, 409 veraltetes `expected_revision` und 422 Anforderungsfehler auf Schemaebene. Ein 409 bedeutet, dass der externe gewünschte Zustand neu berechnet werden muss.

6. Zeichnen Sie Aktualisierungsnachweise auf und vermeiden Sie blinde Wiederholungen.

WARNUNG:

Versuchen Sie nicht blind, denselben PATCH erneut auszuführen. Speichern Sie alte Revision, neue Revision, Validierungstext, Patch-Body-Hash, Antwortstatus, Prüfergenehmigung und die Wiederholungsentscheidung außerhalb von BPM.

Die Profilaktualisierung wird erst nach Validierungs- und Revisionsprüfungen angewendet, mit expliziten Prüfnachweisen für Erfolg oder Konflikt.

Exportieren Sie das Profil oder übergeben Sie Compliance-Metadaten erst, nachdem das zurückgelesene Profil dem genehmigten gewünschten Status entspricht.

Import, Prüfung, Export und Übergabe von Compliance-Metadaten durchführen

Verbinden Sie ein externes Prüf- oder Compliance-Produkt mit dem BPM-Import und -Export API-Schnittstellen, ohne BPM-Metadaten als Firefox-Bereitstellungsinhalte zu behandeln.

Legen Sie `$BPM_BASE_URL` fest und bereiten Sie ein überprüftes Firefox-Quelldokument `policies.json` sowie externe Compliance-Metadaten wie Scanner, Ticket, Prüfer und Entscheidung vor.

Dieser Arbeitsablauf verwendet API-FF-001, API-PROFILE-003, API-VAL-001 und API-FF-002. BPM speichert undurchsichtige Compliance-Metadaten im Profil, exportiertes Firefox-JSON enthält jedoch nur bereitstellbare `policies`.

1. Importieren Sie das überprüfte Quelldokument in ein BPM-Profil.

Anforderungsbeispiel: POST `$BPM_BASE_URL/api/profiles/import/firefox/policies.json` mit JSON `{"name":"reviewed-corp-baseline","schema_version":"release-153","compliance":{"scanner":"external-control","ticket":"SEC-42"},"document":{"policies":{"DisableTelemetry":true}}}`. Antwortbeispiel: 201 ProfileRead.

2. Lesen Sie das erstellte Profil zurück und trennen Sie BPM-Felder von externen Beweisen.

Anforderungsbeispiel: GET `$BPM_BASE_URL/api/profiles/42`. Antwortbeispiel: 200 ProfileRead mit `revision`, `flags` und undurchsichtigem `compliance`. Fehlerbeispiele: 404 oder 422.

3. Überprüfen Sie den normalisierten Richtlinienatz vor dem Export.

Der Prüfer prüft importierte Schlüssel, Schemakanal, Quelldokument-Hash, Compliance-Metadaten und alle BPM-Normalisierungswarnungen. BPM zertifiziert weder das Ergebnis des externen Scanners noch die Firefox-Laufzeitdurchsetzung.

4. Validieren Sie das Bereitstellungsdokument, das exportiert wird.

Anforderungsbeispiel: POST `$BPM_BASE_URL/api/validate/release-153` mit JSON `{"document":{"policies":{"DisableTelemetry":true}}}`. Antwortbeispiel: 200 `{"ok":true}`. Fehlerbeispiele: 400, 404, 422 oder 503.

5. Exportieren Sie das kanonische Firefox-Richtliniendokument.

Anforderungsbeispiel: GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1`. Antwortbeispiel: 200 `application/json` mit Content-Disposition und `policies` der obersten Ebene. Compliance-Metadaten werden nicht exportiert.

6. Schreiben Sie den Übergabedatensatz vor der Downstream-Bereitstellung.

WARNUNG:

Quell-Hash, Profil ID, Revision, Validierungsantwort, Export-Hash, Scannerergebnis, Prüfer und Downstream-Eigentümer außerhalb von BPM speichern. Betrachten Sie den BPM-Export nicht als Beweis dafür, dass Firefox die Richtlinie angewendet hat.

Das externe Produkt erhält ein kanonisches Firefox-Bereitstellungsdokument und einen separaten Compliance-Übergabedatensatz mit klaren Eigentumsgrenzen.

Verwenden Sie Bereitstellungstools außerhalb von BPM, um die exportierte `policies.json` auf verwalteten Firefox-Systemen zu platzieren.

Start eines Steuerungsprodukts mit BPM-Health- und Readiness-Prüfungen

Verwenden Sie die aktuellen Integritätstests als Starttor, bevor ein externes Steuerungsprodukt mit der API-Synchronisierung beginnt.

Die BPM-Quellenbereitstellung wurde bereits gestartet. Konfigurieren Sie das externe Produkt mit `$BPM_BASE_URL`, Zeitüberschreitung, Abfrageintervall, maximaler Wartezeit und der Aktion, die ausgeführt werden soll, wenn das Gate ausfällt.

Dieser Arbeitsablauf verwendet API-HEALTH-001 und API-HEALTH-002. Bei den Sonden handelt es sich lediglich um kleine Vorflugsignale; Sie legen keine Authentifizierung, Autorisierung, Datenbankdiagnose, Schema-Cache-Diagnose, Anforderung Kennungen oder Service-Level-Ziele offen. Dieser - Betriebsanleitung-Satz benennt den zustandsgesteuerten Start, den Dateneigentum, die Wiederherstellung nach Fehlern und die Regel, nicht blind erneut zu versuchen, wenn Beweise veraltet sind.

Die Verfügbarkeit des Dokumentationsassistenten ist von BPM-Health und -Readiness getrennt. In 0.9.3 erhält jede Assistentenfrage den veröffentlichten Hinweis zur Schulung des lokalen Modells; Modell, Suche, RAG-Index und Anfragen an externe Quellen werden nicht gestartet. Diese Grenze blockiert die deterministische Dokumentationssuche nicht und darf kein Gate für Profiländerungen oder API-Synchronisierung sein.

1. Überprüfen Sie die grundlegende Lebendigkeit, bevor Sie die Synchronisierungwarteschlange öffnen.

Anforderungsbeispiel: `GET $BPM_BASE_URL/health`. Antwortbeispiel: `200 {"status":"ok"}`. Transportfehler werden vom externen Produkt behandelt, da für dieses Probe kein BPM-Anwendungsfehlertext deklariert ist.

2. Überprüfen Sie die Bereitschaft vor Profil- oder Validierungsaufrufen.

Anforderungsbeispiel: `GET $BPM_BASE_URL/health/ready`. Antwortbeispiel: `200 {"status":"ready", "ready":true}`. Wenn `ready` nicht `true` ist, halten Sie die Warteschlange und starten Sie keine Mutationen.

3. Zeichnen Sie die Start-Gate-Entscheidung auf.

Speichern Sie Zeitstempel, Basis-URL-Bezeichnung, Zeitüberschreitung, Sondenstatus, beobachteten Körper, Kontrollproduktversion und ausgewählte nächste Aktion. Speichern Sie keine privaten Hosts in Beispielen für freigegebene Betriebsanleitungen.

4. Beginnen Sie mit der schreibgeschützten Bestandsaufnahme, nachdem das Tor passiert ist.

Rufen Sie `GET $BPM_BASE_URL/api/profiles?limit=50` vor Import-, Export-, Validierungs- oder Patch-Vorgängen auf. Dadurch bleibt die anfängliche Synchronisierung beobachtbar und wiederherstellbar.

5. Wechseln Sie zu Mutationsworkflows erst, nachdem schreibgeschützte Aufrufe erfolgreich waren.

Fahren Sie mit der Validierung vor der Anwendung, dem Import, der Überprüfung, dem Export oder dem Export von Arbeitsabläufen gemäß dem externen Produktplan fort. Halten Sie die Wiederholungs- und Backoff-Richtlinie außerhalb von BPM.

6. Fehler geschlossen, wenn das Tor nicht eindeutig ist.

WARNUNG:

Eine erfolgreiche Prüfung reserviert keine Revisionen und garantiert keinen späteren API-Erfolg. Eine fehlgeschlagene oder mehrdeutige Prüfung muss automatisierte Mutationen stoppen, bis ein Bediener Protokolle, Quellbereitstellungsstatus und Netzwerk-/CORS-Grenzen überprüft.

Das externe Steuerungsprodukt startet die Synchronisierung nur, wenn BPM auf den aktuellen Integritäts- und Bereitschafts-Handshake reagiert.

Wenn das Starttor fehlschlägt, verwenden Sie die Themen „Administrator/DevOps“ zur Fehlerbehebung und Quellbereitstellung, anstatt undokumentierte Probe-Annahmen hinzuzufügen.

Wiederherstellung nach Integrationsfehlern und Dokumentation von Prüfnachweisen

Behandeln Sie Validierungs-, Import-/Export-, Bereitschafts- und Revisionsfehler ohne unsichere Wiederholungsversuche oder versteckte Transaktionsannahmen.

Verwenden Sie diesen Arbeitsablauf, wenn ein externes Produkt eine fehlgeschlagene BPM-API-Antwort, eine Zeitüberschreitung, einen unerwarteten Antworttext, veraltete `expected_revision` oder nicht übereinstimmende Exportnachweise erhält.

Dieser Arbeitsablauf verbindet API-HEALTH-001, API-HEALTH-002, API-PROFILE-003, API-PROFILE-005, API-FF-001, API-FF-002 und API-VAL-001. Es definiert Bedienernachweise; BPM bietet keine verteilte Ablaufverfolgung, kein automatisches Rollback, keine Massentransaktionswiederherstellung oder Webhook-Benachrichtigungen.

1. Klassifizieren Sie den Fehler, bevor Sie es erneut versuchen.

Zeichnen Sie auf, ob es sich bei dem Fehler um Integrität/Bereitschaft, Validierung, Import, Export, Lesen, Patch, Konflikt, Zeitüberschreitung oder Transport handelt. Behalten Sie den HTTP-Status bei, z. B. 400, 404, 409, 415, 422 oder 503.

2. Frieren Sie den externen gewünschten Zustand und die BPM-Beobachtung ein.

Anforderungsmethode, Pfad, bereinigter Nutzlast-Hash, Profil ID, beobachtetes `revision`, Schemakanal, Antworttext, Antwortzeit und Kontrollproduktausführung ID speichern. Beziehen Sie niemals Geheimnisse oder private Hostnamen in gemeinsame Beweise ein.

3. Überprüfen Sie den BPM-Zustand erneut und lesen Sie das Profil erneut, wenn der Fehler behebbar ist.

Anforderungsbeispiele: GET `$BPM_BASE_URL/health`, GET `$BPM_BASE_URL/health/ready` und GET `$BPM_BASE_URL/api/profiles/42`. Wenn sich die Profilrevision geändert hat, kehren Sie zum Vergleich und zur Genehmigung zurück.

4. Validierung oder erneuter Export nur nach einer vom Betreiber genehmigten Entscheidung.

Zur Validierung wiederholen Sie POST `$BPM_BASE_URL/api/validate/release-153` mit demselben Kandidatendokument. Wiederholen Sie für den Export GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?pretty=1` und vergleichen Sie den exportierten Dokument-Hash.

5. Mutationswiederholungen bei veralteter Revision oder mehrdeutigen Beweisen stoppen.

Wenn `expected_revision` 409 zurückgibt, sich der Export-Hash unterscheidet oder sich Validierungsfehler geändert haben, setzen Sie den alten Lauf nicht fort. Öffnen Sie einen neuen externen Lauf mit neuer Lektüre, Vergleich und Genehmigung des Prüfers.

6. Veröffentlichen Sie den Prüfdatensatz im externen Aufzeichnungssystem.

WARNUNG:

BPM speichert den Profilstatus, speichert jedoch keinen vollständigen Integrations-Audit-Trail. Das externe Produkt muss Eigentümer, Entscheidung, Ticket, Wiederholungsanzahl, Stoppgrund, alte und neue Revisionen, Hashes und Bedienernotizen aufzeichnen.

Fehler können nur durch explizite Beweise, neue Lesevorgänge und vom Bediener genehmigte Wiederholungsentscheidungen behoben werden, nicht durch blinde Wiedergabe.

Verknüpfen Sie nach der Wiederherstellung den Prüfdatensatz mit der erfolgreichen Profilrevision oder markieren Sie den Integrationslauf als gestoppt.

Fehler beheben und Betriebsbereitschaft bewerten

Fehlerbehebung bei fehlgeschlagenem Start und nicht erreichbaren Sonden

Diagnostizieren Sie einen von der Quelle ausgeführten BPM-Prozess, der nicht startet oder Gesundheits- und Bereitschaftsprüfungen nicht beantwortet.

Verwenden Sie dieses Verfahren, wenn `make dev, uvicorn app.main:app --reload --port 8000, GET /health` oder `GET /health/ready` während der Quellbereitstellung oder des Integrationsstarts fehlschlägt.

Diese Diagnose hält Prozess-, Umgebungs- und Sondennachweise getrennt. Es wird keine Systemd-Einheit, kein Windows-Dienst, kein Reverse-Proxy, kein TLS-Endpunkt und keine Produktionsneustarttrichtlinie erstellt.

1. Erfassen Sie den fehlgeschlagenen Befehl und das Vordergrundprotokoll.

Führen Sie den genauen Befehl aus oder kopieren Sie ihn und zeichnen Sie dann `stdout` und `stderr` vom Vordergrundterminal auf. Fügen Sie `make dev, uvicorn app.main:app --reload --port 8000`, die Python-Traceback-Zusammenfassung, das Arbeitsverzeichnis und die Quellrevision von `git rev-parse --short HEAD` hinzu.

2. Überprüfen Sie den konfigurierten Host und Port, bevor Sie den Code ändern.

Überprüfen Sie `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_DATABASE_URL` und `BPM_DOCUMENTATION_SITE_DIR`. Konzentrierte Wiederholung: `curl -fs http://127.0.0.1:8000/health` und `curl -fs http://127.0.0.1:8000/health/ready`.

3. Trennen Sie den Prozessstart von der Probe-Erreichbarkeit.

Wenn der Prozess nicht ausgeführt wird, beheben Sie zunächst Abhängigkeits-, Umgebungs- oder Migrationsnachweise. Wenn der Prozess ausgeführt wird, die Sonden jedoch nicht erreichbar sind, überprüfen Sie die lokale Firewall, die Bindungsadresse, die WSL-Portweiterleitung und ob die Anforderung auf `127.0.0.1`, `0.0.0.0` oder eine Windows-Hostroute abzielt.

4. Bewahren Sie Benutzerdaten bei der Beweiserhebung.

Nur Protokolle, Umgebungsnamen und Probe-Antworten kopieren. Bearbeiten Sie generierte Artefakte, Datenbankzeilen, Schema-Cache-Dateien oder Quellcode nicht im Rahmen einer routinemäßigen Wiederherstellung. Löschen Sie `data/bpm.db` nicht, damit der Startvorgang erfolgreich ist.

5. Wählen Sie den kleinsten Wiederholungs- und Eskalationspfad.

Fokussierte Wiederholung: für Produktrauch, nur, wenn Dokumentationsausgabe beteiligt ist, und nur für Dokumentationsverträge.

WARNUNG:

Wenn Probes nach Konfigurations- und Prozessprüfungen weiterhin nicht erreichbar sind, stoppen Sie automatisierte Integrationsläufe und eskalieren Sie mit Befehlen, Protokollen, Konfigurationsnamen und Probe-Beweisen.

Der Bediener kann einen fehlgeschlagenen Prozess, eine nicht erreichbare Sonde und eine Bereitstellungsgrenze unterscheiden, ohne Benutzerdaten zu verändern.

Nachdem die Tests bestanden wurden, kehren Sie zum Thema „Integritätsgesteuerter Start“ oder „wiederverwendbare API-Beispiele“ zurück, bevor Sie mutierende Integrationen ausführen.

Schema-Cache- und API-Validierungsfehler beheben

Diagnostizieren Sie einen nicht verfügbaren Schema-Cache, einen unbekannten Schema-Kanal und Validierungsfehler bei Firefox-Richtlinien, ohne die generierten Schema-Artefakte zu patchen.

Verwenden Sie dieses Verfahren, wenn POST `$BPM_BASE_URL/api/validate/release-153`, POST `$BPM_BASE_URL/api/validate/beta-999` oder die benutzerseitige Validierung 400, 404, 422 oder 503 zurückgibt; Ein Nicht-Objektdokument kann 200 `ok=false` mit Expected object with policy mappings zurückgeben. Die

Die BPM-Validierung hängt von den gepflegten Schemakanälen und der lokalen Schema-/Cache-Konfiguration ab. Die Diagnose zeichnet auf, ob es sich bei dem Problem um eine Anforderungsform, ein unbekanntes Profil, ungültige Firefox-Richtliniendaten oder einen nicht verfügbaren Schema-Cache handelt.

1. Zeichnen Sie die genaue Validierungsanfrage und -antwort auf.

Speichern Sie bereinigten Anforderungstext-Hash, Schemakanal, HTTP-Status, detail, message, error und issues. Behalten Sie den Kandidaten `policies.json` außerhalb der Protokolle, wenn er Kundenwerte enthält.

2. Zuerst werden Anforderungsformfehler klassifiziert.

Ein Nicht-Objekt-document oder ein fehlendes `document.policies` ist eine Bedienereingabe und kein Schema-Cache-Defekt; Das aktuelle BPM kann dies als 200 `ok=false` melden. Konzentrierte Wiederholung: POST `$BPM_BASE_URL/api/validate/release-153` mit `{"document":{"policies":{"DisableTelemetry":true}}}`.

3. Schemakanal- und Cachefehler klassifizieren.

Ein Profil wie `beta-999` kann 404 zurückgeben; Ein nicht verfügbares registriertes Schema kann 503 zurückgeben. Überprüfen Sie `BPM_SCHEMA_BASE_URL`, `BPM_SCHEMA_CACHE_DIR` und `BPM_SCHEMA_HTTP_TIMEOUT` vor einer umfassenderen Neuerstellung.

4. Verwenden Sie gepflegte Aktualisierungs- und Validierungsprüfungen.

Gezielte Wiederholung: für API-Dokumentationsdrift oder Produktschematests, wenn sich das API-Verhalten geändert hat. Bearbeiten Sie `app/schemas/cache` oder generierte Firefox-Skelette nicht von Hand.

5. Stoppen Sie unsichere Wiederherstellungen und sichern Sie Beweise.

Wenn sich der Status derselben gültigen Anforderung über mehrere Ausführungen hinweg ändert, behalten Sie Antworttexte, Quellrevision, Cache-Pfad und Umgebungsvariablen bei.

WARNUNG:

Bearbeiten Sie generierte Schemaartefakte oder Datenbankzeilen nicht als Routinewiederherstellung. Trennen Sie Produktmängel von Bediener-Nutzlastfehlern.

Der Operator hat einen klassifizierten Validierungsfehler mit einer kleinsten Wiederholung und keiner versteckten Schemamutation.

Nach erfolgreicher Validierung kehren Sie zu den Arbeitsabläufe „Validieren vor dem Anwenden“ oder „Importieren, Überprüfen und Exportieren“ zurück.

Fehlerbehebung bei Firefox-Import- und Exportfehlern

Diagnostizieren Sie JSON-Import-, Multipart-Import- und Firefox-Richtlinien-Exportfehler unter Beibehaltung der Quell- und exportierten Beweise.

Verwenden Sie dieses Verfahren, wenn POST

`$BPM_BASE_URL/api/profiles/import/firefox/policies.json` oder GET

`$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json` 400, 404, 409, 415 oder 422 zurückgibt.

Die Import-/Export-Fehlerbehebung trennt die Gültigkeit des Quelldokuments, die Form mehrteiliger Felder, doppelte Profilnamen, den Zugriff auf archivierte Profile und den kanonischen Firefox-Export von nachgelagerten Bereitstellungstools.

1. Identifizieren Sie den Vorgang und den Inhaltstyp.

Erfassen Sie, ob die Anfrage `JSON FirefoxPoliciesJsonImportRequest`, `multipart/form-data` mit `file=@policies.json` oder ein Export mit `download=1`, `pretty=1` oder `include_deleted=true` war.

2. Überprüfen Sie das Firefox-Quelldokument, bevor Sie den Import erneut versuchen.

Konzentrierte Wiederholung: POST `$BPM_BASE_URL/api/validate/release-153` mit demselben `document.policies`. Wenn die Validierung fehlschlägt, korrigieren Sie das Quelldokument außerhalb von BPM, bevor Sie es erneut importieren.

3. Importspezifische Bedienerfehler prüfen.

Wählen Sie für 409 einen neuen synthetischen Profilnamen oder archivieren/wiederherstellen Sie absichtlich. Überprüfen Sie für 415 die Felder `application/json` oder mehrteilige Uploads. Zeichnen Sie für 422 Fehler auf Feldebene auf und schreiben Sie den Profilspeicher nicht direkt neu.

4. Exportspezifischen Status prüfen.

Für 404 lesen Sie noch einmal GET `$BPM_BASE_URL/api/profiles/42?include_deleted=true`. Entscheiden Sie für archivierte Profile, ob `include_deleted=true` angemessen ist. Vergleichen Sie den Hash des exportierten Dokuments mit dem überprüften Quell-Hash. BPM-Profilmetadaten und Compliance-Metadaten werden nicht exportiert.

5. Wiederherstellungs- und Stoppbedingungen aufzeichnen.

Fokussierte Wiederholung: für dokumentierte Beispiele.

WARNUNG:

Versuchen Sie es nicht blind erneut, wenn die Import-/Exportnachweise nicht übereinstimmen. Quell-Hash speichern, Hash exportieren, Profil ID, Revision, HTTP-Status und Betreiberentscheidung.

Der Bediener kann identifizieren, ob der Fehler auf Quell-JSON, mehrteilige Felder, Profilebenszyklusstatus oder Exportnachweise zurückzuführen ist.

Fügen Sie nach der Wiederherstellung die Beweise dem externen Kontrollprodukt-Audit-Datensatz bei.

Datenbank- und Speicherprobleme beheben

Lokale SQLite-, Speicherpfad-, Sicherungs- und Profilsichtbarkeitsprobleme diagnostizieren, ohne Datenbankzeilen direkt zu bearbeiten.

Verwenden Sie dieses Verfahren, wenn Profile verschwinden, Schreibvorgänge fehlschlagen, Migrationsnachweise unklar sind, Sicherungsprüfsummen nicht übereinstimmen oder `BPM_DATABASE_URL` auf einen unerwarteten Speicherort verweist.

Die aktuelle Quellbereitstellung verwendet lokalen Speicher wie `sqlite+aiosqlite:///./data/bpm.db` und Repository-eigene Dateien unter `data/`, Schema-Cache-Pfade und Dokumentations-Site-Pfade. In BPM gibt es keine verwaltete Backup- oder Wiederherstellungsorchestrierung.

1. Bestätigen Sie die aktive Datenbank-URL und das Arbeitsverzeichnis.

Zeichnen Sie `BPM_DATABASE_URL`, das aktuelle Verzeichnis, die Quellrevision und ob der Prozess vom Linux-Dateisystem oder von `/mnt/c` ausgeführt wird, auf. Gehen Sie nicht davon aus, dass zwei Shells auf denselben `data/bpm.db` zeigen.

2. Überprüfen Sie die Migrations- und Profil-API-Beweise, bevor Sie Speicherdateien öffnen.

Konzentrierte Wiederholung: `alembic upgrade head`, dann `curl -fsS http://127.0.0.1:8000/api/profiles?limit=50`. Wenn die API erwartete Profile zurückgibt, kann das Symptom ein Filter, ein Lebenszyklus oder ein Browserstatus sein.

3. Überprüfen Sie die Sicherungsnachweise vor der destruktiven Wiederherstellung.

Überprüfen Sie die Datensätze `mkdir -p backups`, `cp data/bpm.db backups/bpm-pre-update.db` und `sha256sum backups/bpm-pre-update.db`. Bewahren Sie Exporte kritischer `policies.json`-Profile als separate Beweise auf.

4. Vermeiden Sie direkte Änderungen an Datenbankzeilen.

Bearbeiten Sie SQLite-Zeilen, generierte Artefakte oder Schema-Cache-Dateien nicht als routinemäßige Wiederherstellung. Verwenden Sie stattdessen dokumentierte Profilarchivierungs-, Wiederherstellungs-, Import-, Export- oder Update-from-Source-Verfahren.

5. Eskalieren Sie mit Speichernachweisen, wenn die Wiederherstellung unsicher ist.

Geben Sie Datenbank-URL, Dateipfad, Prüfsumme, Migrationsbefehl, API-Antwort und Sicherungs-/Exportnachweise an.

WARNUNG:

Wenn keine verifizierte Sicherung vor dem Ausfall vorhanden ist, stoppen Sie, bevor Sie `data/bpm.db` löschen oder ersetzen. BPM bietet keine Point-in-Time-Wiederherstellung.

Der Operator verfügt über eine Speicherdiagnose, die Benutzerdaten bewahrt und den Datenbankstatus von UI oder Filtersymptomen trennt.

Nachdem der Speicher stabil ist, führen Sie den Zustand/die Bereitschaft und den am wenigsten betroffenen API-Arbeitsablauf erneut aus.

Fehlerbehebung bei WSL-Netzwerken und veralteten Abhängigkeiten

Diagnostizieren Sie Windows 10/11 anhand von WSL-Zugriffsproblemen, der Platzierung des Linux-Dateisystems und veralteten Python-Abhängigkeiten nach Quelländerungen.

Verwenden Sie dieses Verfahren, wenn Windows einen von der WSL gehosteten BPM-Prozess nicht erreichen kann, Tests innerhalb der WSL, aber nicht von Windows aus funktionieren, die Dateileistung unter `/mnt/c` instabil ist oder Abhängigkeiten nach `git checkout <approved-new-ref>` veraltet erscheinen.

Bei der WSL-Fehlerbehebung bleiben der Windows-Browserzugriff, die Linux-Prozessbindung, die Dateisystemplatzierung und die Aktualisierung der Python-Umgebung getrennt. BPM stellt in 0.9.1 keinen nativen Windows-Dienst oder Installer bereit.

1. Bestätigen Sie, wo sich der Quell-Checkout und die virtuelle Umgebung befinden.

Bevorzugen Sie das Linux-Dateisystem für den Checkout und `.venv`. Zeichnen Sie auf, ob sich Dateien unter `/mnt/c` befinden, was sich auf Leistung und Berechtigungen auswirken kann. Mischen Sie während der Diagnose nicht mehrere Überprüfungen.

2. Überprüfen Sie die WSL-Prozessbindung und die Windows-Route.

Überprüfen Sie `BPM_HOST="0.0.0.0"`, `BPM_PORT="8000"`, `curl -fsS http://127.0.0.1:8000/health` innerhalb der WSL und den Browserzugriff von Windows. Wenn Windows ausfällt, notieren Sie die Firewall- und Portweiterleitungsnachweise.

3. Abhängigkeiten nach Quellverschiebung oder Revisionsänderungen aktualisieren.

Konzentrierte Wiederholung: `source .venv/bin/activate`, `python -m pip install --upgrade pip` und `pip install -e ".[dev]"`. Wenn weiterhin Abhängigkeitsfehler bestehen, notieren Sie die Paketnamen und die Quellrevision, anstatt die installierten Pakete manuell zu bearbeiten.

4. Trennen Sie veraltete Abhängigkeitssymptome von Schema- oder Datenbankproblemen.

Wenn der Import/Export oder die Validierung nach der Aktualisierung der Abhängigkeit fehlschlägt, verwenden Sie Schema/Cache und Import/Export-Fehlerbehebung. Wenn sich die Profilsichtbarkeit ändert, verwenden Sie die Datenbank-/Speicher-Fehlerbehebung.

5. Stoppen Sie unsichere native Windows-Annahmen.

Erstellen Sie keine undokumentierten Windows-Dienste, verschieben Sie die Datenbank nicht zwischen Dateisystemen und beanspruchen Sie kein natives Installationsverhalten.

WARNUNG:

Wenn das WSL-Netzwerk nicht eindeutig ist, stoppen Sie externe Integrationen und eskalieren Sie mit der WSL-Distribution, dem Checkout-Pfad, der Bindungsadresse, der Windows-URL und den Testausgaben.

Der Bediener kann WSL-Routing, Dateisystemplatzierung und Abhängigkeitsaktualisierungsfehler unterscheiden, ohne den Produktspeicher zu ändern.

Nachdem der WSL-Zugriff stabil ist, kehren Sie zur Überprüfung der Quellbereitstellung und zum zustandsgesteuerten Start zurück.

Nicht verfügbares Dokumentationsportal beheben

Sammeln Sie den sichtbaren Zustand von /help/ und die Bereitstellungsdaten, wenn das installierte Dokumentationsportal nicht verfügbar ist.

Verwenden Sie dieses Verfahren, wenn /help/ einen fehlenden, veralteten, inkompatiblen, unvollständigen oder nicht gefundenen Zustand meldet.

Das Dokumentationsportal ist Teil der BPM-Oberfläche. Seine Verfügbarkeit hängt davon ab, dass das installierte Artefakt zur laufenden BPM-Version passt.

1. Portal-URL und sichtbaren Zustand erfassen

Das Dokumentationsportal ist Teil der BPM-Oberfläche. Seine Verfügbarkeit hängt davon ab, dass das installierte Artefakt zur laufenden BPM-Version passt.

2. Laufende BPM-Instanz und Dokumentationseinstellung bestätigen

Das Dokumentationsportal ist Teil der BPM-Oberfläche. Seine Verfügbarkeit hängt davon ab, dass das installierte Artefakt zur laufenden BPM-Version passt.

3. Abhängige Dokumentationsaktionen anhalten

WARNUNG:

Behandeln Sie ein fehlendes oder inkompatibles Portal nicht als funktionsfähige Hilfe und bearbeiten Sie installierte Dateien nicht von Hand.

4. Nachweise an den BPM-Bereitstellungsverantwortlichen eskalieren

Das Dokumentationsportal ist Teil der BPM-Oberfläche. Seine Verfügbarkeit hängt davon ab, dass das installierte Artefakt zur laufenden BPM-Version passt.

5. Portal nach der Wiederherstellung prüfen

Das Dokumentationsportal ist Teil der BPM-Oberfläche. Seine Verfügbarkeit hängt davon ab, dass das installierte Artefakt zur laufenden BPM-Version passt.

Das Dokumentationsportal ist Teil der BPM-Oberfläche. Seine Verfügbarkeit hängt davon ab, dass das installierte Artefakt zur laufenden BPM-Version passt.

Portal nach der Wiederherstellung prüfen.

Netzwerkexposition und Proxy-Bereitschaft prüfen

Bereiten Sie Hinweise zur Netzwerkexposition und Reverse-Proxy-Fragen vor, ohne nicht unterstützte TLS- oder Proxy-Rezepte als BPM-Verhalten zu veröffentlichen.

Die aktuelle Bewertung der Quellbereitschaft eines einzelnen Knotens ist abgeschlossen und der Betreiber weiß, ob Benutzer BPM über Loopback, eine WSL-weitergeleitete Adresse, einen Laborhost oder einen externen Proxy erreichen.

BPM kann auf einem konfigurierten Host und Port ausgeführt werden, die BPM-Dokumentation bietet jedoch keinen offiziellen Reverse-Proxy, keine TLS-Terminierung, kein Authentifizierungs-Gateway, keine Header-Richtlinie oder kein Ratenbegrenzungsrezept.

1. Wählen Sie die kleinste Netzwerkexposition, die der aktuellen Bewertung entspricht.

127.0.0.1 bevorzugen. Verwenden Sie `BPM_HOST="0.0.0.0"` nur, wenn die WSL-Weiterleitung oder ein kontrolliertes Labornetzwerk dies erfordert, und notieren Sie, warum `BPM_PORT="8000"` erreichbar ist.

2. Notieren Sie sich Fragen zur Proxy-Bereitschaft, bevor Sie einen externen Proxy hinzufügen.

Beantworten Sie, wer Eigentümer von TLS-Zertifikaten ist, Hostnamen-Routing, Anforderungsgrößenbeschränkungen, Zeitüberschreitungswerte, CORS-Ursprünge bis `BPM_CORS_ALLOW_ORIGINS`, weitergeleitete Header, Zugriffsprotokolle und Reaktion auf Vorfälle. Diese Fragen können jetzt vorbereitet werden; Die offiziellen TLS/Proxy-Rezepte sind geplant, aber noch nicht implementiert.

3. Stellen Sie sicher, dass die Sonden weiterhin über die ausgewählte Grenze hinweg funktionieren.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Wenn in einem Labor ein externer Proxy verwendet wird, zeichnen Sie sowohl direkte Quellprüfungen als auch Proxy-seitige Prüfungen auf. Behandeln Sie eine Proxy-200-Antwort nicht als Beweis für die BPM-Authentifizierung, -Autorisierung oder HA.

4. Markieren Sie nicht unterstützte Netzwerkansprüche explizit.

WARNUNG:

BPM liefert keine Paketdienste, offiziellen TLS/Proxy-Rezepte, unterstütztes HA-Clustering, WAF-Regeln, Anforderungssignierung, Ratenbegrenzung, SSO, OAuth/OIDC, rollenbasierte Zugriffskontrolle oder Produktionshärtung. Bewahren Sie diese als externe Kontrollen oder aufgeschobene Anforderungen auf.

Der Netzwerkhinweis trennt, was jetzt konfiguriert werden kann, von Reverse-Proxy- und TLS-Entscheidungen, die extern bleiben oder zurückgestellt werden.

Verwenden Sie das Thema „Überwachung, Sicherung und Aktualisierungsfenster“, bevor ein gemeinsames Bewertungsfenster beginnt.

Überwachungseingänge, Sicherungsnachweise und Aktualisierungsfenster planen

Bereiten Sie den aktuellen Betriebsnachweis für eine von der Quelle ausgeführte Instanz vor und vermeiden Sie gleichzeitig Behauptungen über verwaltete Überwachung, offizielle Wiederherstellungsautomatisierung oder fortlaufende Upgrades.

Die Instanz hat die Überprüfung der Quellbereitschaft und der Netzwerkgefährdung bestanden und ein einzelner Administrator oder DevOps-Eigentümer ist für das Bewertungsfenster verantwortlich.

Bis Verteilungen und Laufzeitdienstentscheidungen vorliegen, sollte sich die Betriebsplanung auf beobachtbare Eingaben, manuelle Sicherungs- und Exportnachweise sowie explizite Aktualisierungsfenster für einen Quellknoten konzentrieren.

1. Listen Sie die Überwachungseingänge auf, die heute vorhanden sind.

Prozessstatus, Vordergrund-Terminalausgabe, `stdout`, `stderr`, `GET /health`, `GET /health/ready`, ausgewählte API-Beispielergebnisse, Dokumentationszugriff über `/help/` und die Quellrevision von `git rev-parse --short HEAD` aufzeichnen.

2. Sammeln Sie Backups und exportieren Sie Beweise vor dem Fenster.

```
mkdir -p backups
cp data/bpm.db backups/bpm-pre-update.db
sha256sum backups/bpm-pre-update.db
```

Exportieren Sie außerdem repräsentative Firefox-Dokumente wie `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json` und speichern Sie die resultierenden `policies.json`-Beweise mit Prüfsummen.

3. Planen Sie ein Stop-the-World-Update-Fenster für Quelländerungen.

Planen Sie für 0.9.1 die Ausfallzeit eines Knotens vor `git fetch --tags`, Abhängigkeitsaktualisierung, `alembic upgrade head`, und . Rollierende Upgrades sind geplant, aber noch nicht implementiert.

4. Erfassen Sie Lücken als Blocker, nicht als versteckte Annahmen.

WARNUNG:

BPM bietet keine verwaltete Überwachung, keine automatisierte Backup-Planung, keine offizielle Wiederherstellungsautomatisierung, keine Point-in-Time-Wiederherstellung, keine verwalteten Geheimnisse, keine Token-Rotation oder keine fortlaufende Upgrade-Orchestrierung. Verwenden Sie manuelle Beweise und Stoppbedingungen aus dem Update-Betriebsanleitung.

Der Betriebsplan enthält Überwachungseingaben, Sicherungs-/Exportnachweise und ein realistisches Aktualisierungsfenster für den aktuellen Quellbetrieb.

Hängen Sie diesen Nachweis an den HA- und Deferred-Boundary-Datensatz an, damit bei späteren Verteilungsarbeiten dieselbe Checkliste wiederverwendet werden kann.

BPM-Dokumentationsassistent

Den Dokumentationsassistenten in BPM 0.9.3 betreiben

Prüfen Sie die veröffentlichte Grenze des Schulungshinweises, ohne BPM oder die deterministische Suche vom Chat abhängig zu machen.

Verwenden Sie einen unterstützten BPM-0.9.3-Quellcode-Checkout und die Grundressourcen aus [Mindestsystemanforderungen für BPM](#).

In dieser Version ist der Chat nur eine lokalisierte Leseroberfläche. Er startet oder installiert kein Modell, ruft keine Dokumentation ab, verwendet keinen E5-Index, erzeugt keine Antwort, zeigt keine Zitate und kontaktiert keine externen Quellen. Die deterministische Dokumentationssuche bleibt getrennt und verfügbar.

1. Starten Sie BPM mit dem normalen unterstützten Quellcode-Workflow.

Fügen Sie dem Start von BPM in 0.9.3 keine Modell-, Index-, Embedding- oder Anbieter-Konfiguration hinzu.

2. Öffnen Sie das Dokumentationsportal in der Sprache des Lesers.

Prüfen Sie, dass sich der Assistent unten rechts öffnen lässt und die normalen Suchsteuerelemente unverändert bleiben.

3. Senden Sie eine Frage zu BPM.

Im Transkript muss der lokalisierte Hinweis erscheinen, dass die Schulung des lokalen Modells läuft und die Funktion in einer späteren Version verfügbar wird.

4. Senden Sie eine themenfremde Frage.

Es wird derselbe Hinweis erwartet; der Chat ist in dieser Version kein allgemeiner Assistent.

5. Ermitteln Sie die aktuelle Antwort mit der deterministischen Dokumentationssuche.

Suche, Filter, Navigation und Ergebnisreihenfolge bleiben der unterstützte Informationsweg.

Der Assistent zeigt den lokalisierten Schulungshinweis; BPM und die deterministische Suche bleiben unabhängig nutzbar.

Installieren Sie für 0.9.3 kein Modell und erstellen Sie keine RAG-Generation. Melden Sie abweichenden Hinweis, Gebietsschema oder Suchgrenze als Dokumentationsfehler.

Die Grenze des Dokumentationsassistenten in BPM

0.9.3 pflegen

Halten Sie den lokalisierten Schulungshinweis und die Grenze zur deterministischen Suche bei der Dokumentationspflege korrekt.

Halten Sie die freigegebene BPM-Quellcode-Revision und den Dokumentations-Publikationsworkflow bereit.

In 0.9.3 gibt es kein lokales Modell, kein Modellartefakt, keine RAG-Generation, keinen Embedding- oder Zitatspeicher und keinen Anbieter externer Quellen zu pflegen. Erzeugen Sie keinen Wartungszustand für nicht gelieferte Funktionen.

1. Übernehmen Sie die freigegebene BPM-Revision und bauen Sie die Dokumentation neu.

Verwenden Sie den normalen Dokumentationsworkflow, einschließlich `make docs-install-dev` für eine Entwicklungsumgebung.

2. Öffnen Sie die veröffentlichte Dokumentation in allen unterstützten Sprachen.

Prüfen Sie den Einstieg des Assistenten und die unabhängige deterministische Suche.

3. Senden Sie in jedem Gebietsschema eine Frage.

Prüfen Sie den lokalen Schulungshinweis sowie das Fehlen von Antwort, Quelle, Zitat, externer Behauptung und Modellinstallation.

4. Führen Sie in 0.9.3 keine Befehle zur Modellinstallation oder RAG-Generierung aus.

Sie gehören zu einem späteren Release und beheben keinen Schulungshinweis.

5. Dokumentieren Sie die Dokumentationsrevision und das Prüfergebnis.

Behandeln Sie eine Abweichung zwischen Hinweis und Suchgrenze als Dokumentationsfehler.

Das veröffentlichte Portal behält einen lokalisierten Hinweis im Chat und einen getrennten Pfad zu aktuellen Informationen über die deterministische Suche.

Bewahren Sie keine Leserfragen als Wartungsartefakt auf. Prüfen Sie Modell- und Retrieval-Betrieb erst, wenn ein späteres Release ihn ausdrücklich liefert.